

Corporación Unificada Nacional de Educación Superior — CUN

Educación Virtual · Actividad de Aprendizaje (ACA) · Semanas 3, 4, 5 y 6

Diseño de la arquitectura de software de un Sistema de Gestión Académica (SGA)

Requerimientos, modelos del sistema, trazabilidad y análisis crítico con apoyo de Google AI Studio

Asignatura	Arquitectura de Software
Estudiante	Jhonnathan Brandon Galvis Landinez
Docente	Alexander Calderón Martínez
Grupo / NRC	55536
Fecha	8 de noviembre de 2026

Colombia · 2026

Contenido

1. Introducción y descripción del problema 3
2. Objetivos 3
3. Alcance, actores, supuestos, restricciones y reglas de negocio 5
4. Requerimientos funcionales y no funcionales: listado y análisis 8
5. Arquitectura seleccionada: descripción y justificación 11
6. Modelo del sistema 15
6.1 Modelo de casos de uso 15
6.2 Modelo de componentes / vista lógica 18
6.3 Modelo de despliegue 20
6.4 Modelo de datos conceptual 22
6.5 Vistas de secuencia 24
7. Matriz de trazabilidad de requerimientos 26
8. Decisiones arquitectónicas, atributos de calidad y análisis de riesgos 28
9. Evidencias del uso de Google AI Studio 33
10. Video pitch y repositorio 49
11. Conclusiones y recomendaciones 49
12. Referencias (APA 7.ª edición) 51

Los números de página son de referencia. Archivos fuente de los diagramas: carpeta `diagramas/` del repositorio (formatos `.dot`, `.mmd`, `.svg` y `.png`).

1. Introducción y descripción del problema

Una institución de educación superior administra hoy la información de estudiantes, docentes, programas, asignaturas, grupos, matrículas, calificaciones y horarios en archivos y aplicaciones independientes. Esta fragmentación produce tres problemas concretos que el caso describe: **duplicidad de datos** (el mismo estudiante o asignatura existe en varias fuentes con valores distintos), **demoras en la consulta** (no hay un único punto de acceso ni índices sobre los datos) y **pérdida de trazabilidad** (no es posible saber quién modificó una nota o una matrícula, ni cuándo).

Este informe propone la arquitectura de software de un **Sistema de Gestión Académica (SGA)** web que centraliza esa información, controla el acceso por roles y deja registro auditable de las operaciones sensibles. El trabajo no implementa el sistema completo: produce una arquitectura sustentada en los 12 requerimientos funcionales (RF) y 12 no funcionales (RNF) del enunciado, los modelos del sistema exigidos (casos de uso, componentes, despliegue, datos y secuencia), una matriz de trazabilidad con su método de verificación y un análisis crítico de riesgos.

La propuesta se construyó con apoyo de **Google AI Studio**, usado como asistente para generar alternativas y borradores; cada sugerencia se contrastó contra los requerimientos y se aceptó, modificó o descartó con una justificación explícita (sección 9). La decisión de estilo arquitectónico se argumenta frente al tamaño, la complejidad y los atributos de calidad del sistema, no por recomendación de la herramienta.

> **Capacidades propuestas vs. implementadas.** Todo lo descrito en este documento es una **propuesta de diseño**. Ningún componente ha sido implementado ni probado en producción; las métricas (por ejemplo, 3 s con 300 usuarios) son objetivos de diseño con un método de verificación propuesto en la matriz de trazabilidad, no resultados medidos.

2. Objetivos

2.1 Objetivo general

Diseñar y modelar la arquitectura de software de un Sistema de Gestión Académica web, seguro, mantenible, escalable y accesible, que satisfaga los requerimientos funcionales y no funcionales planteados y cuyas decisiones queden justificadas y trazadas hasta los componentes que las implementan.

2.2 Objetivos específicos

1. Analizar el contexto del caso y especificar los RF y RNF de forma verificable, identificando actores, supuestos, restricciones y reglas de negocio.
2. Seleccionar y justificar un estilo arquitectónico comparando al menos tres alternativas (capas, monolito modular y microservicios) frente a los atributos de calidad.
3. Elaborar los modelos de casos de uso, componentes, despliegue, datos conceptual y secuencia con nombres y relaciones consistentes entre sí.

4. Construir una matriz de trazabilidad que relacione cada RF y RNF con componentes, modelos, decisiones y un método de verificación.
5. Identificar riesgos, puntos únicos de falla, amenazas de seguridad y límites de escalabilidad, y documentar las decisiones arquitectónicas principales.
6. Usar Google AI Studio de forma crítica, documentando prompts, respuestas y los ajustes realizados por el autor.

3. Alcance, actores, supuestos, restricciones y reglas de negocio

3.1 Alcance

Dentro del alcance

- Autenticación, roles y permisos (administrador, coordinador académico, docente, estudiante).
- Gestión de estudiantes, docentes, programas, planes de estudio y asignaturas.
- Configuración de periodos, grupos, cupos, docentes, horarios y aulas o enlaces.
- Matrícula con validación de cupo y duplicidad.
- Registro, actualización y publicación de calificaciones.
- Consulta del estudiante, reportes básicos, búsqueda y filtros.
- Auditoría de eventos relevantes, respaldo y recuperación.
- API REST documentada para integración futura.

Fuera del alcance

- Pagos, liquidación de matrícula financiera y facturación.
- Admisiones e inscripción de aspirantes.
- Aula virtual / LMS (contenidos, entregas, foros).
- Aplicación móvil nativa (se cubre con web adaptable).
- Generación de certificados con firma digital.
- Migración detallada de los datos existentes (se deja como riesgo y recomendación).
- Implementación completa del sistema (el enunciado no la exige).

3.2 Actores

Actor	Tipo	Responsabilidades y objetivos en el SGA
Administrador	Primario	Gestiona usuarios, roles y permisos; registra y desactiva estudiantes y docentes; consulta la bitácora de auditoría; genera reportes institucionales.
Coordinador académico	Primario	Define programas, planes y asignaturas; configura periodos, grupos, cupos, horarios y docentes asignados; realiza o asiste matrículas; genera reportes por programa y periodo.
Docente	Primario	Consulta los grupos que dicta; registra, actualiza y publica calificaciones de sus estudiantes matriculados.
Estudiante	Primario	Se matricula en la ventana habilitada; consulta asignaturas matriculadas, horario y calificaciones publicadas.
Plataformas institucionales	Secundario (futuro)	Consumen la API REST (por ejemplo, LMS, financiera, SSO institucional). No forman parte del alcance actual pero condicionan la decisión de interoperabilidad (RNF-08).
Reloj del sistema / tareas programadas	Secundario	Dispara respaldos, cierre automático de ventanas de matrícula y procesamiento de la cola de eventos de auditoría.

3.3 Supuestos

ID	Supuesto	Impacto si es falso
S-01	El escenario de referencia es de hasta 300 usuarios concurrentes (RNF-02), con picos al abrir la matrícula.	Si la institución es mucho mayor, se escala horizontalmente la API y se separan lecturas (sección 8.5).
S-02	La escala de calificación es 0,0 a 5,0 y una nota final se compone de componentes con porcentajes que suman 100 %.	Cambia solo la regla de validación del módulo Calificaciones.
S-03	La matrícula la puede realizar el coordinador o el propio estudiante dentro de una ventana habilitada por periodo.	Si solo la hace el coordinador, se retira el permiso al rol estudiante sin cambiar la arquitectura.
S-04	El sistema se despliega en un proveedor de nube con al menos dos zonas de disponibilidad y servicio de almacenamiento de objetos.	En un centro de datos propio se requiere replicar manualmente la redundancia; sube el riesgo de disponibilidad.
S-05	El equipo de desarrollo domina C#/.NET y Angular.	El estilo arquitectónico se mantiene; solo cambia la pila tecnológica (por ejemplo, Spring Boot o NestJS).
S-06	La autenticación es propia del SGA en esta fase; la integración con SSO institucional (OIDC) es futura.	Si ya existe un proveedor de identidad, el módulo IAM delega el login y conserva solo la autorización.

3.4 Restricciones

- **R-01 Acceso web:** la solución debe ser accesible desde navegadores recientes de uso común y adaptarse a pantallas de computador y móvil (RNF-06, RNF-12).
- **R-02 Seguridad mínima obligatoria:** HTTPS en todas las comunicaciones, contraseñas sin texto plano y control por roles (RNF-01).
- **R-03 Interfaz de integración:** preferiblemente API REST documentada (RNF-08).
- **R-04 Normativa:** tratamiento de datos personales conforme a la Ley 1581 de 2012 de Colombia (refuerza RNF-11).
- **R-05 Académica:** no se exige implementar el sistema completo; el entregable es la arquitectura y sus modelos. No se usan datos personales reales ni credenciales en prompts, capturas o repositorios.
- **R-06 Presupuesto:** contexto institucional con presupuesto limitado; se privilegian componentes de código abierto y servicios administrados de bajo costo.

3.5 Reglas de negocio

ID	Regla	Origen
RN-01	Un estudiante no puede tener dos matrículas activas en la misma asignatura dentro del mismo periodo.	RF-07
RN-02	Solo se matricula en un grupo si el periodo está en estado *matrícula abierta* y el grupo tiene cupo disponible (`cupos_ocupados < cupo_maximo`).	RF-06, RF-07

ID	Regla	Origen
RN-03	Un docente solo registra o actualiza calificaciones de los grupos que tiene asignados y mientras el periodo no esté cerrado.	RF-08, RF-02
RN-04	Las notas están en el rango 0,0–5,0 y los porcentajes de los componentes de un grupo suman 100 %.	RF-08, S-02
RN-05	El estudiante solo consulta sus propios datos y únicamente las calificaciones marcadas como publicadas.	RF-09, RNF-11
RN-06	Estudiantes y docentes no se eliminan físicamente: se desactivan, para conservar historial y trazabilidad.	RF-03, RF-04, RF-12
RN-07	Modificar una calificación ya publicada exige un motivo y queda registrada en auditoría con el valor anterior y el nuevo.	RF-08, RF-12
RN-08	Un grupo no puede tener cupo máximo menor que el número de matriculados activos.	RF-06, RNF-07

4. Requerimientos funcionales y no funcionales: listado y análisis

Los requerimientos del enunciado se conservan con su identificador original. Para cada uno se agregó el actor principal, la prioridad (MoSCoW: **M** = debe, **S** = debería) y un **criterio de aceptación verificable**, porque varios enunciados originales son correctos pero no comprobables tal como están redactados.

4.1 Requerimientos funcionales (RF)

ID	Nombre	Actor(es)	Prior.	Criterio de aceptación verificable
RF-01	Autenticación	Todos	M	Con credenciales válidas se obtiene una sesión (token de acceso de 15 min y refresh token); con inválidas se responde 401 sin revelar cuál dato falló; tras 5 intentos fallidos en 15 min la cuenta se bloquea temporalmente; el cierre de sesión invalida el refresh token.
RF-02	Roles y permisos	Administrador	M	Existen al menos los 4 roles; cada endpoint exige un permiso; un usuario sin permiso recibe 403; el cambio de rol surte efecto en la siguiente renovación de token (≤ 15 min) y queda auditado.
RF-03	Gestión de estudiantes	Administrador, personal autorizado	M	Se registra, consulta, actualiza y desactiva un estudiante; el documento es único; desactivar no borra el historial (RN-06).
RF-04	Gestión de docentes	Administrador, personal autorizado	M	Igual que RF-03 para docentes; un docente desactivado no puede asignarse a grupos nuevos.
RF-05	Programas y asignaturas	Coordinador	M	Se crean y actualizan programas, planes y asignaturas con código único, nombre, créditos y semestre/nivel; una asignatura puede pertenecer a varios planes.
RF-06	Grupos y periodos	Coordinador	M	Se configura un periodo con fechas y estado, y grupos con cupo, docente, horario y aula o enlace; se rechaza un cupo menor que los matriculados (RN-08).
RF-07	Matrícula académica	Coordinador, Estudiante	M	Se matricula si hay cupo y no hay duplicidad; con 100 solicitudes simultáneas sobre un grupo de 30 cupos se crean exactamente 30 matrículas y 70 respuestas 409; una misma solicitud repetida (mismo `Idempotency-Key`) no crea dos registros.
RF-08	Registro de calificaciones	Docente	M	El docente registra y actualiza notas solo en sus grupos (RN-03) y dentro del rango (RN-04); dos ediciones concurrentes sobre la misma nota producen una 409 en la segunda (control optimista).
RF-09	Consulta de resultados	Estudiante	M	El estudiante ve sus asignaturas, horario y solo notas publicadas propias; acceder a datos de otro estudiante devuelve 403/404.
RF-10	Reportes	Administrador, Coordinador	S	Se generan reportes de matriculados, grupos, asignaturas y resultados filtrando por periodo y programa, en pantalla y exportables a CSV/PDF.
RF-11	Búsqueda y filtros	Usuarios autorizados	M	Cada listado permite buscar por identificador o nombre y filtrar por programa, periodo o estado, con paginación; solo se devuelven registros que el rol puede ver.

ID	Nombre	Actor(es)	Prior.	Criterio de aceptación verificable
RF-12	Auditoría	Sistema / Administrador	M	Cada inicio de sesión, cambio de rol, matrícula y modificación de nota genera un evento con usuario, fecha UTC, acción, entidad, registro, valores antes/después e IP; los eventos no se pueden editar ni borrar desde la aplicación.

4.2 Requerimientos no funcionales (RNF)

Los RNF se clasificaron según los atributos de calidad del modelo ISO/IEC 25010 y se expresaron como escenarios medibles (estímulo, respuesta y medida), siguiendo la práctica de Bass et al. (2021).

ID	Atributo ISO 25010	Escenario / métrica objetivo
RNF-01	Seguridad (confidencialidad, autenticidad)	100 % del tráfico en HTTPS (TLS 1.2+, HSTS); contraseñas con hash Argon2id con sal; ningún endpoint de negocio accesible sin token y permiso.
RNF-02	Eficiencia de desempeño	Con 300 usuarios concurrentes en el escenario de referencia, p95 ≤ 3 s en consultas comunes (definidas como: listado paginado, consulta de horario, consulta de notas, búsqueda por documento).
RNF-03	Fiabilidad (disponibilidad)	≥ 99,5 % mensual, equivalente a un máximo aproximado de 3 h 39 min de indisponibilidad no programada al mes.
RNF-04	Eficiencia / mantenibilidad (escalabilidad)	Agregar una instancia de API no requiere cambios de código (API sin estado); el modelo de datos soporta crecimiento por periodos sin rediseño.
RNF-05	Mantenibilidad (modularidad, capacidad de prueba)	Ningún módulo accede a tablas de otro; dependencias solo por interfaces públicas, verificadas por pruebas de arquitectura; cobertura de pruebas del dominio ≥ 70 %.
RNF-06	Usabilidad / accesibilidad	Cumplimiento WCAG 2.2 nivel AA en las funciones principales: contraste ≥ 4,5:1, etiquetas en formularios, operación completa por teclado.
RNF-07	Fiabilidad / integridad	Ninguna matrícula sobrepasa el cupo ni se duplica ante solicitudes simultáneas o fallos a mitad de la operación (transacción ACID).
RNF-08	Compatibilidad (interoperabilidad)	API REST versionada (`/api/v1`) con contrato OpenAPI 3 publicado y errores en formato `application/problem+json`.
RNF-09	Fiabilidad (recuperabilidad)	Respaldo base diario y archivado continuo de WAL; RPO ≤ 15 min y RTO ≤ 4 h; restauración probada al menos una vez por periodo académico.
RNF-10	Mantenibilidad (trazabilidad del diseño)	Cada RF/RNF aparece en la matriz con al menos un componente y un método de verificación; cada decisión tiene un ADR.
RNF-11	Seguridad (confidencialidad, privacidad)	Acceso a datos por necesidad del rol y por propiedad del registro; datos personales minimizados en logs; cumplimiento de la Ley 1581 de 2012.
RNF-12	Compatibilidad / portabilidad	Funciona en las dos últimas versiones de Chrome, Edge, Firefox y Safari, en anchos de 360 px a 1920 px.

4.3 Análisis de los requerimientos

Requerimientos que dirigen la arquitectura. No todos los requerimientos pesan igual en el diseño. Los que más condicionan la estructura son: RNF-07 (consistencia de matrículas y notas ante concurrencia), RNF-01 y RNF-11 (seguridad y privacidad por rol), RNF-03 (disponibilidad de 99,5 %), RNF-05 (bajo acoplamiento) y RF-12 (auditoría transversal). Los demás se satisfacen con decisiones locales dentro de un módulo.

Ambigüedades detectadas y cómo se resolvieron:

- **RNF-02 «consultas comunes»** no estaba definido; se acotó a cuatro operaciones concretas para poder medirlo con pruebas de carga.
- **RF-07** no indica quién matricula; se asumió coordinador y estudiante (S-03), controlado por permisos.
- **RF-08 y RF-09** mencionan notas «publicadas», lo que implica un estado de publicación; se agregó el caso de uso UC-09 y el atributo `publicada`.
- **RNF-09** no fija tiempos de recuperación; se propusieron RPO ≤ 15 min y RTO ≤ 4 h, coherentes con el margen mensual de RNF-03, y quedan como decisión a validar con la institución.
- **RF-03 «desactivar»** sugiere borrado lógico; se formalizó en RN-06 para no romper la trazabilidad (RF-12).

Tensiones entre requerimientos. La auditoría completa (RF-12) y la consistencia transaccional (RNF-07) agregan trabajo a cada escritura, lo que compite con el rendimiento (RNF-02). Se resuelve escribiendo el evento en la misma transacción (tabla *outbox*) y procesándolo de forma asíncrona. La privacidad (RNF-11) también compite con la búsqueda abierta (RF-11): los filtros se aplican en el servidor según rol y propiedad, nunca solo en la interfaz.

5. Arquitectura seleccionada: descripción y justificación

5.1 Alternativas evaluadas

Se compararon tres estilos con una escala de 1 (débil) a 5 (fuerte) frente a los atributos de calidad del caso. La evaluación considera el tamaño del sistema (8 módulos funcionales, un solo dominio), el equipo esperado (3 a 5 desarrolladores) y la carga de referencia (300 usuarios concurrentes).

Criterio	Capas tradicional	Monolito modular en capas	Microservicios	Comentario
Consistencia de matrícula/notas (RNF-07)	5	5	2	Con una base de datos, una transacción ACID basta; en microservicios exige sagas y compensaciones.
Mantenibilidad y bajo acoplamiento (RNF-05)	2	4	5	Las capas puras tienden a mezclar dominios; el monolito modular impone fronteras por módulo.
Escalabilidad (RNF-04)	3	4	5	300 usuarios no requieren escalar módulos por separado; basta escalar instancias sin estado.
Disponibilidad (RNF-03)	3	4	4	Ambos alcanzan 99,5 % con redundancia; los microservicios añaden más puntos de falla de red.
Complejidad operativa	5	4	1	Microservicios requieren orquestación, descubrimiento, trazas distribuidas y varias bases.
Costo de infraestructura	5	4	2	Contexto de presupuesto limitado (R-06).

Criterio	Capas tradicional	Monolito modular en capas	Microservicios	Comentario
Facilidad de evolución futura	2	4	5	Un módulo con fronteras claras puede extraerse como servicio si la carga lo justifica.
Total	**25**	**29**	**24**	Se selecciona el **monolito modular en capas**.

5.2 Estilo seleccionado: monolito modular con arquitectura limpia por módulo

El SGA se construye como **una sola aplicación desplegable** (una API REST y una SPA) dividida internamente en **ocho módulos de negocio** con fronteras explícitas. Dentro de cada módulo se aplican capas al estilo de la arquitectura limpia (Martin, 2017): *dominio* (entidades y reglas), *aplicación* (casos de uso), *infraestructura* (persistencia y adaptadores) y *presentación* (controladores REST). Las dependencias apuntan hacia el dominio, lo que permite probar las reglas sin base de datos.

Reglas de modularidad que hacen que el estilo funcione en la práctica:

1. Cada módulo es dueño de su **esquema** en la base de datos (`iam`, `personas`, `catalogo`, `oferta`, `matriculas`, `calificaciones`, `auditoria`). Ningún módulo consulta tablas de otro.
2. La comunicación entre módulos se hace solo por **interfaces públicas** síncronas (por ejemplo, `IOfertaQuery`) o por **eventos de dominio** asíncronos (por ejemplo, `MatriculaRegistrada`).
3. Las reglas anteriores se verifican automáticamente con **pruebas de arquitectura** en la integración continua, para evitar que el monolito se degrade.
4. El módulo de **Reportes** es de solo lectura y puede consultar vistas de lectura diseñadas para él, conectadas a la réplica de la base de datos.

Por qué no microservicios. El sistema tiene un solo dominio con reglas fuertemente acopladas en la operación crítica (matrícula: cupo del grupo + historial del estudiante + auditoría). Distribuir esa operación obligaría a coordinar consistencia entre servicios sin que la carga lo justifique. Siguiendo la recomendación de comenzar con un monolito bien modularizado y extraer servicios solo cuando haya evidencia (Fowler, 2015; Newman, 2019), la arquitectura deja preparada esa evolución sin pagar hoy su costo.

Por qué no capas tradicionales. Una arquitectura de tres capas sin módulos tiende a producir un «servicio de negocio» que conoce todas las tablas, lo que contradice RNF-05 y dificulta las pruebas aisladas.

5.3 Pila tecnológica propuesta y relación con los requerimientos

Elemento	Tecnología	Justificación (requerimientos)
Cliente web	Angular (SPA) con diseño adaptable y componentes accesibles	Tipado fuerte, guards de ruta por rol, soporte de accesibilidad y diseño responsive (RNF-06, RNF-12). Coherente con S-05.
Backend / API	ASP.NET Core (C#), API REST con OpenAPI	Alto rendimiento por instancia, middleware nativo de autenticación, autorización por políticas y rate limiting (RNF-01, RNF-02, RNF-08).
Persistencia	PostgreSQL + EF Core	Transacciones ACID, bloqueo de filas (`SELECT … FOR UPDATE`), índices únicos parciales, replicación y recuperación a un punto en el tiempo (RNF-07, RNF-09).
Caché	Redis	Caché de catálogos poco cambiantes (programas, asignaturas) y contadores de rate limiting (RNF-02).
Contenedores	Docker sobre servicio administrado de contenedores	Despliegue repetible, varias instancias sin estado y autoescalado (RNF-03, RNF-04).
Observabilidad	OpenTelemetry + servicio de logs/métricas	Medir p95, errores y disponibilidad para verificar RNF-02 y RNF-03.
Integración continua	GitHub Actions	Pruebas unitarias, de arquitectura, de integración, análisis de seguridad y despliegue automatizado (RNF-05, RNF-10).

La pila es reemplazable: el estilo arquitectónico y los modelos no dependen de ella. Por ejemplo, Spring Boot + MySQL o NestJS + PostgreSQL cumplirían el mismo rol.

5.4 Tácticas arquitectónicas por atributo de calidad

Atributo	Tácticas aplicadas
Seguridad (RNF-01, RNF-11)	TLS extremo a extremo; WAF; JWT de corta vida + refresh token rotativo en cookie *HttpOnly, Secure, SameSite=Strict*; RBAC por permisos granulares; autorización por propiedad del recurso (el estudiante solo accede a lo suyo); Argon2id; secretos en gestor de secretos; bloqueo por intentos fallidos; consultas parametrizadas (ORM); cabeceras CSP y HSTS.
Rendimiento (RNF-02)	Paginación obligatoria; índices por los campos de filtro de RF-11; caché de catálogos; reportes pesados en el worker y contra la réplica; compresión HTTP; SPA servida por CDN.
Disponibilidad (RNF-03)	Redundancia activa de la API (2 instancias en zonas distintas); health checks y retiro automático de instancias enfermas; base de datos con réplica en espera y conmutación por error; despliegues sin caída (rolling).
Escalabilidad (RNF-04)	API sin estado (sesión en el token); escalado horizontal por CPU/latencia; separación de lecturas hacia réplica; posibilidad de extraer módulos.
Integridad (RNF-07)	Transacción única por caso de uso; bloqueo pesimista del grupo al matricular; índice único parcial como segunda barrera; control de concurrencia optimista (columna `version`) en notas; claves de idempotencia.
Mantenibilidad (RNF-05)	Módulos con fronteras verificadas por pruebas de arquitectura; inyección de dependencias; dominio sin dependencias de infraestructura.

Atributo	Tácticas aplicadas
Interoperabilidad (RNF-08)	REST con OpenAPI 3, versionado en la URL, errores estandarizados (RFC 9457 *problem details*).
Recuperabilidad (RNF-09)	Respaldo base diario + WAL continuo a almacenamiento de objetos en otra región, cifrado y versionado; procedimiento de restauración documentado y ensayado.

5.5 Interfaces principales de la API REST (v1)

Endpoint	Permiso / rol	Propósito (RF)
`POST /auth/login` · `POST /auth/refresh` · `POST /auth/logout`	Público / autenticado	Inicio, renovación y cierre de sesión (RF-01)
`GET|PUT /usuarios/{id}/roles`	Administrador	Asignar roles y permisos (RF-02)
`GET|POST /estudiantes` · `GET|PUT /estudiantes/{id}` · `PATCH /estudiantes/{id}/estado`	Administrador, autorizado	CRUD y desactivación (RF-03, RF-11)
`GET|POST /docentes` · `PUT /docentes/{id}` · `PATCH /docentes/{id}/estado`	Administrador, autorizado	CRUD y desactivación (RF-04, RF-11)
`/programas` · `/programas/{id}/planes` · `/asignaturas`	Coordinador	Catálogo académico (RF-05)
`/periodos` · `/grupos` · `/grupos/{id}/horarios`	Coordinador	Oferta académica (RF-06)
`POST /matriculas` · `PATCH /matriculas/{id}/anular`	Coordinador, Estudiante (propia)	Matrícula (RF-07)
`GET /grupos/{id}/calificaciones` · `PUT /calificaciones/{id}` · `POST /grupos/{id}/calificaciones/publicar`	Docente del grupo	Registro y publicación (RF-08)
`GET /me/matriculas` · `GET /me/horario` · `GET /me/calificaciones`	Estudiante	Consulta de resultados (RF-09)
`GET /reportes/{tipo}?periodo=&programa=&formato=`	Administrador, Coordinador	Reportes (RF-10)
`GET /auditoria?usuario=&accion=&desde=&hasta=`	Administrador	Consulta de bitácora (RF-12)

Todas las rutas llevan el prefijo `/api/v1`. Los listados aceptan `q`, `page`, `pageSize` y filtros por `programa`, `periodo` y `estado` (RF-11).

6. Modelo del sistema

6.1 Modelo de casos de uso

El diagrama muestra los cuatro actores principales y los 13 casos de uso que inician. UC-14 y UC-15 (en gris) no los inicia un actor: son comportamientos incluidos que el sistema ejecuta siempre dentro de otros casos de uso.

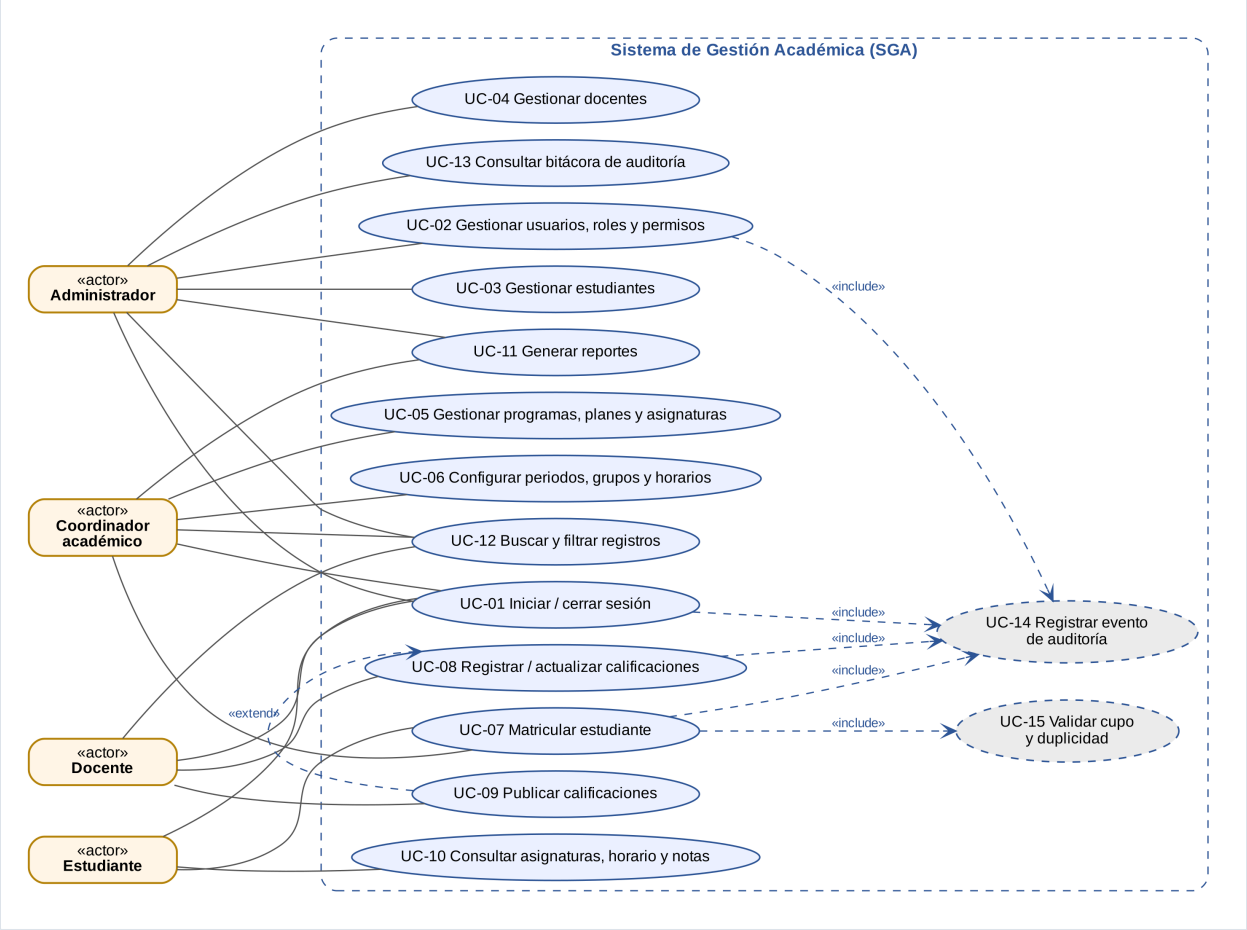


Figura 1. Modelo de casos de uso del SGA. Leyenda: línea continua = asociación actor–caso de uso; flecha discontinua = relación «include» / «extend».

Descripción breve de los casos de uso

ID	Caso de uso	Actor(es)	Descripción	RF
UC-01	Iniciar / cerrar sesión	Todos	Autentica con credenciales individuales; emite y revoca tokens; registra el evento.	RF-01
UC-02	Gestionar usuarios, roles y permisos	Administrador	Crea usuarios, asigna y retira roles, define permisos de cada rol.	RF-02
UC-03	Gestionar estudiantes	Administrador	Registra, consulta, actualiza y desactiva estudiantes.	RF-03

ID	Caso de uso	Actor(es)	Descripción	RF
UC-04	Gestionar docentes	Administrador	Registra, consulta, actualiza y desactiva docentes.	RF-04
UC-05	Gestionar programas, planes y asignaturas	Coordinador	Mantiene el catálogo académico con códigos, créditos y semestres.	RF-05
UC-06	Configurar periodos, grupos y horarios	Coordinador	Abre periodos, crea grupos con cupo, docente, horario y aula/enlace.	RF-06
UC-07	Matricular estudiante	Coordinador, Estudiante	Inscribe a un estudiante en un grupo validando cupo y duplicidad.	RF-07
UC-08	Registrar / actualizar calificaciones	Docente	Ingresa o corrige notas por componente de evaluación.	RF-08
UC-09	Publicar calificaciones	Docente	Hace visibles al estudiante las notas de un componente (extiende UC-08).	RF-08, RF-09
UC-10	Consultar asignaturas, horario y notas	Estudiante	Muestra la información académica propia y las notas publicadas.	RF-09
UC-11	Generar reportes	Administrador, Coordinador	Reportes filtrados por periodo y programa, exportables.	RF-10
UC-12	Buscar y filtrar registros	Usuarios autorizados	Búsqueda paginada con filtros según el módulo y el rol.	RF-11
UC-13	Consultar bitácora de auditoría	Administrador	Filtra eventos por usuario, acción y rango de fechas.	RF-12
UC-14	Registrar evento de auditoría	Sistema	Guarda quién, cuándo, qué acción y qué registro; incluido en UC-01, 02, 07 y 08.	RF-12
UC-15	Validar cupo y duplicidad	Sistema	Aplica RN-01 y RN-02 dentro de la transacción de matrícula.	RF-07

Especificación del caso de uso crítico UC-07 — Matricular estudiante

Actores	Estudiante (matrícula propia) o Coordinador académico (en nombre de un estudiante).
Precondiciones	Usuario autenticado con permiso `matricula.crear`; periodo en estado *matrícula abierta*; estudiante activo.
Flujo principal	1. El actor busca la asignatura y elige un grupo con horario. 2. El sistema muestra cupos disponibles. 3. El actor confirma. 4. El sistema bloquea el grupo, valida cupo (RN-02) y no duplicidad (RN-01). 5. El sistema crea la matrícula, incrementa los cupos ocupados y registra el evento de auditoría en la misma transacción. 6. El sistema confirma y actualiza el horario del estudiante.
Flujos alternos	4a. Sin cupo → mensaje «Grupo sin cupo» y sugerencia de otros grupos de la misma asignatura. 4b. Ya matriculado en la asignatura → mensaje «Ya está matriculado». 3a. Doble clic o reintento de red → la clave de idempotencia devuelve la matrícula ya creada. 1a. Periodo cerrado → la opción no se muestra y la API responde 409.
Postcondiciones	Matrícula activa; `cupos_ocupados` incrementado en 1; evento `MatriculaRegistrada` en bitácora.

Especificación del caso de uso crítico UC-08 — Registrar / actualizar calificaciones

Actor	Docente.
Precondiciones	Docente asignado al grupo; periodo no cerrado; componentes de evaluación configurados con porcentajes que suman 100 %.
Flujo principal	1. El docente abre la planilla del grupo. 2. Ingresa notas por estudiante y componente. 3. El sistema valida rango 0,0–5,0 y pertenencia al grupo. 4. Guarda con control de versión y registra auditoría con valor anterior y nuevo. 5. Opcionalmente publica (UC-09).
Flujos alternos	3a. Nota fuera de rango → error por campo. 4a. Otra sesión modificó la nota → 409 y recarga de la planilla. 4b. Nota ya publicada → exige motivo del cambio (RN-07).
Postcondiciones	Nota guardada con nueva versión; evento `CalificacionModificada` en bitácora.

6.2 Modelo de componentes / vista lógica

La vista lógica muestra cómo se organiza el monolito modular. La SPA se comunica solo con la API por HTTPS. Toda petición pasa por el pipeline de middleware (autenticación y autorización centralizadas) antes de llegar a los módulos, de modo que ningún módulo puede omitir el control de acceso. Los módulos se comunican por interfaces de consulta (flechas azules) y publican eventos de dominio que alimentan la auditoría (flecha discontinua dorada).

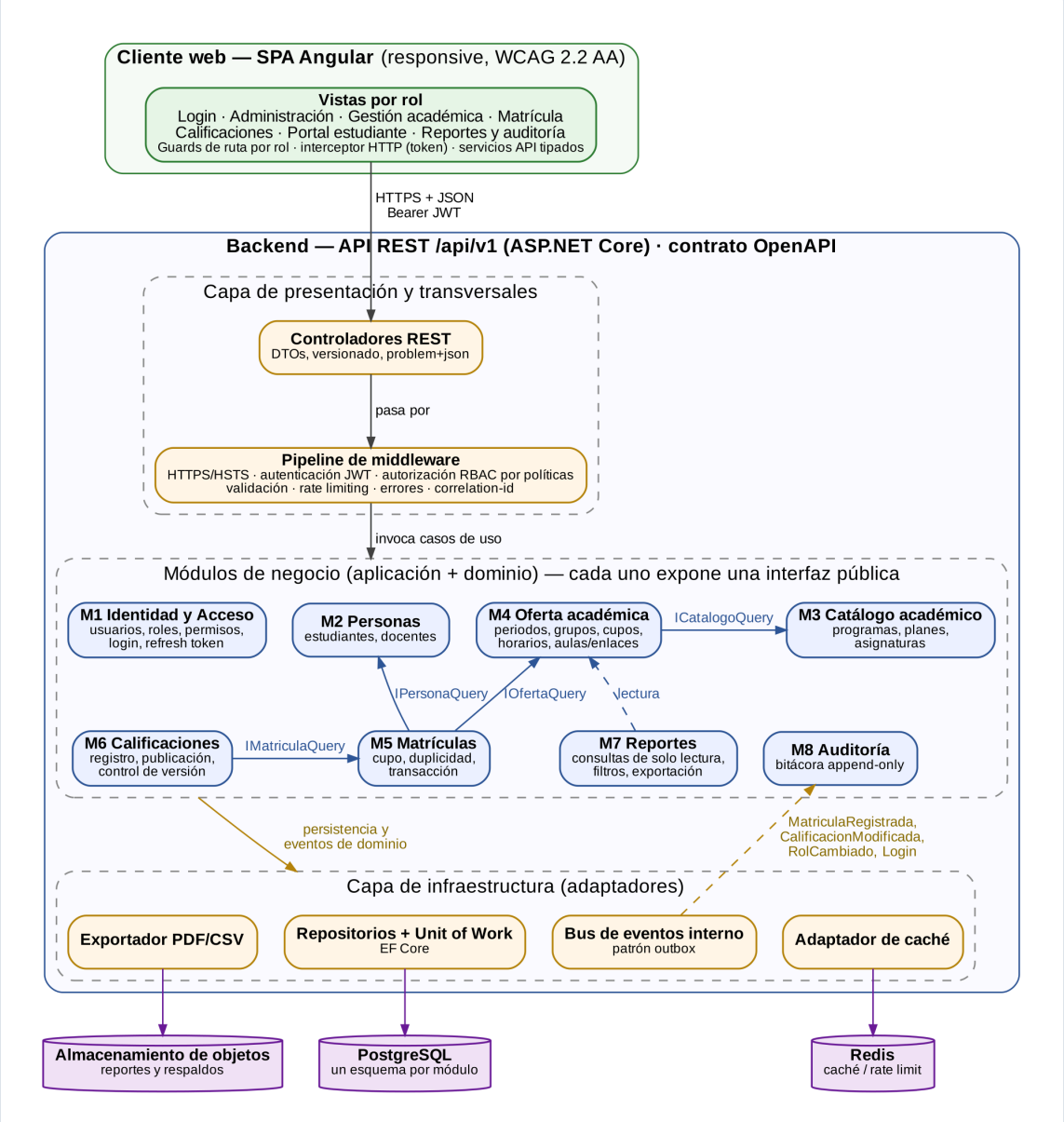


Figura 2. Vista lógica de componentes del SGA. Leyenda: verde = cliente; azul = módulos de negocio; dorado = presentación e infraestructura; morado = almacenamiento; línea discontinua = comunicación asíncrona o de solo lectura.

Responsabilidades, interfaces y dependencias de los módulos

Módulo	Responsabilidad	Interfaz pública / eventos	Depende de
M1 Identidad y Acceso	Usuarios, credenciales (Argon2id), emisión y revocación de tokens, roles, permisos, bloqueo por intentos.	REST `/auth`, `/usuarios`; políticas de autorización consumidas por el middleware; eventos `LoginRealizado`, `RolCambiado`.	Ninguno (módulo base).
M2 Personas	Datos de estudiantes y docentes, estado activo/inactivo, vínculo con usuario.	`IPersonaQuery` (estado, programa del estudiante).	M1 (vincular usuario).
M3 Catálogo académico	Programas, planes de estudio, asignaturas, créditos, semestres.	`ICatalogoQuery`.	Ninguno.
M4 Oferta académica	Periodos y su estado, grupos, cupos, docentes asignados, horarios, aulas o enlaces.	`IOfertaQuery` (grupo, cupos, periodo, docente del grupo).	M3, M2.
M5 Matrículas	Caso de uso de matrícula: bloqueo, validación de cupo y duplicidad, anulación.	`IMatriculaQuery`; evento `MatriculaRegistrada` / `MatriculaAnulada`.	M4, M2.
M6 Calificaciones	Componentes evaluativos, registro y actualización de notas, publicación, control de versión.	Evento `CalificacionModificada`, `CalificacionesPublicadas`.	M5 (inscritos), M4 (docente del grupo).
M7 Reportes	Consultas de solo lectura con filtros; exportación CSV/PDF en segundo plano.	REST `/reportes`.	Vistas de lectura de M3–M6 (réplica).
M8 Auditoría	Persistir eventos de forma inmutable (solo inserción) y consultarlos.	REST `/auditoria`; suscriptor de todos los eventos.	Ninguno (solo recibe eventos).

Organización de autenticación y autorización. M1 emite el token con el identificador del usuario y sus permisos. El middleware valida firma, vigencia y permiso requerido por cada endpoint (autorización por políticas). La verificación de *propiedad* (por ejemplo, que el estudiante consulte sus propias notas o que el docente edite solo sus grupos) se hace en la capa de aplicación del módulo, porque depende de datos del dominio.

6.3 Modelo de despliegue

El despliegue separa tres zonas de red. Solo la CDN/WAF y el balanceador son públicos; la API, el worker y los datos viven en redes privadas sin dirección pública. La API se ejecuta en al menos dos contenedores sin estado, ubicados en zonas de disponibilidad distintas; la base de datos tiene un primario y una réplica en espera en otra zona.

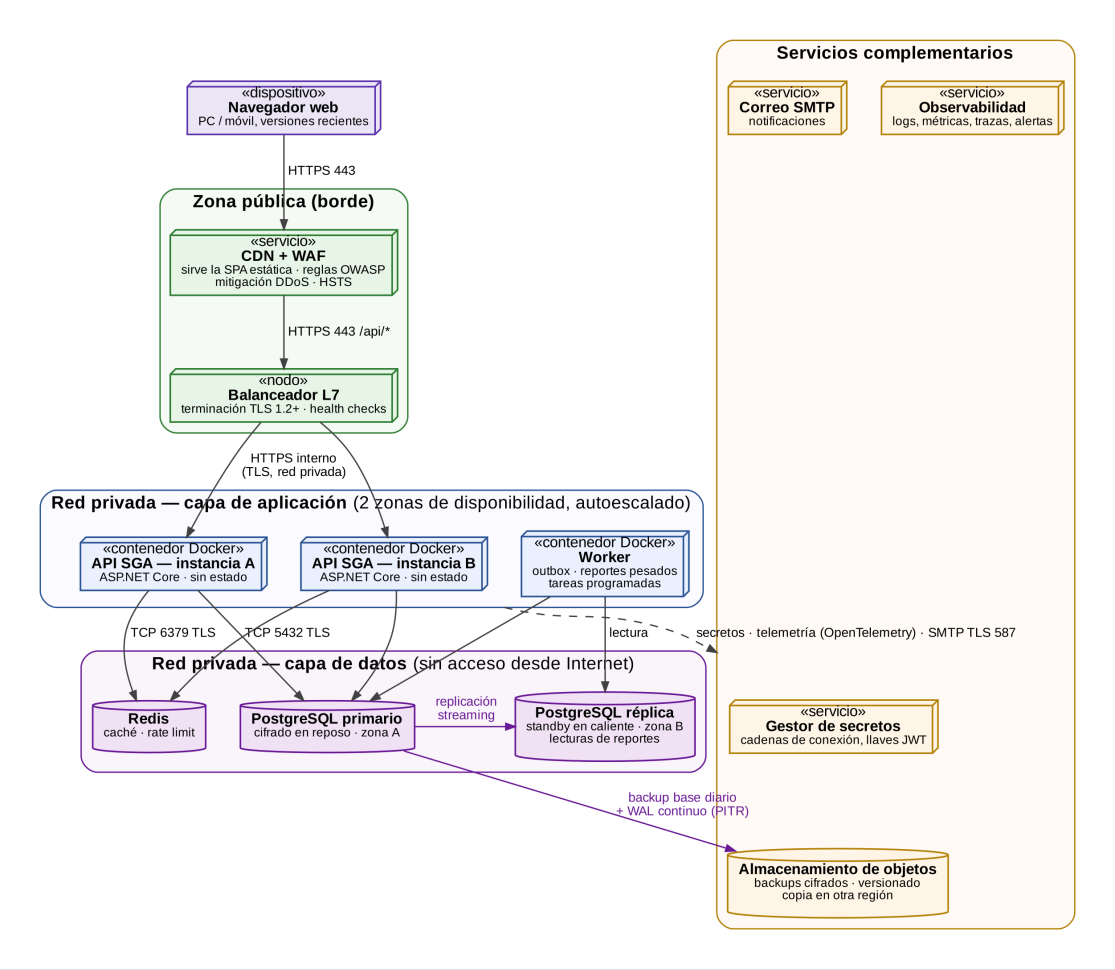


Figura 3. Modelo de despliegue del SGA. Las etiquetas de las conexiones indican protocolo y puerto.

Nodos, canales y controles de seguridad

Nodo / servicio	Función	Controles de seguridad y disponibilidad
Navegador web	Ejecuta la SPA.	Solo HTTPS; tokens de acceso en memoria y refresh token en cookie *HttpOnly*; CSP.
CDN + WAF	Sirve archivos estáticos de la SPA y reenvía `/api/*`.	Reglas OWASP (inyección, XSS), mitigación DDoS, HSTS, caché de estáticos.
Balanceador L7	Distribuye peticiones entre instancias.	Terminación TLS 1.2+ y re-cifrado TLS hacia la API; redirección 80 → 443; health checks cada 10 s; retira instancias que fallan.

Nodo / servicio	Función	Controles de seguridad y disponibilidad
API SGA (A y B)	Ejecuta el monolito modular.	Sin estado; en red privada; credenciales desde gestor de secretos; rate limiting; autoescalado.
Worker	Procesa outbox, reportes pesados, tareas programadas.	Mismo artefacto que la API con otro punto de entrada; lee de la réplica.
PostgreSQL primario / réplica	Datos transaccionales.	Conexión TLS; cifrado en reposo; usuario de BD con mínimo privilegio por esquema; replicación por streaming; conmutación por error.
Redis	Caché y contadores de rate limiting.	Red privada, TLS y autenticación; si falla, el sistema sigue funcionando sin caché (degradación).
Almacenamiento de objetos	Respaldos y archivos exportados.	Cifrado, versionado, bloqueo contra borrado durante la retención, copia en otra región.
Observabilidad	Logs, métricas, trazas, alertas.	Logs sin contraseñas ni datos personales completos; alertas de disponibilidad y latencia p95.

6.4 Modelo de datos conceptual

El modelo incluye las entidades pedidas en el enunciado (Usuario, Rol, Estudiante, Docente, Programa, Asignatura, PeriodoAcadémico, Grupo, Matrícula y Calificación) y las entidades de apoyo necesarias para que las relaciones muchos a muchos y la auditoría sean coherentes (Permiso, PlanEstudio, Horario, EventoAuditoría y las tablas de asociación).

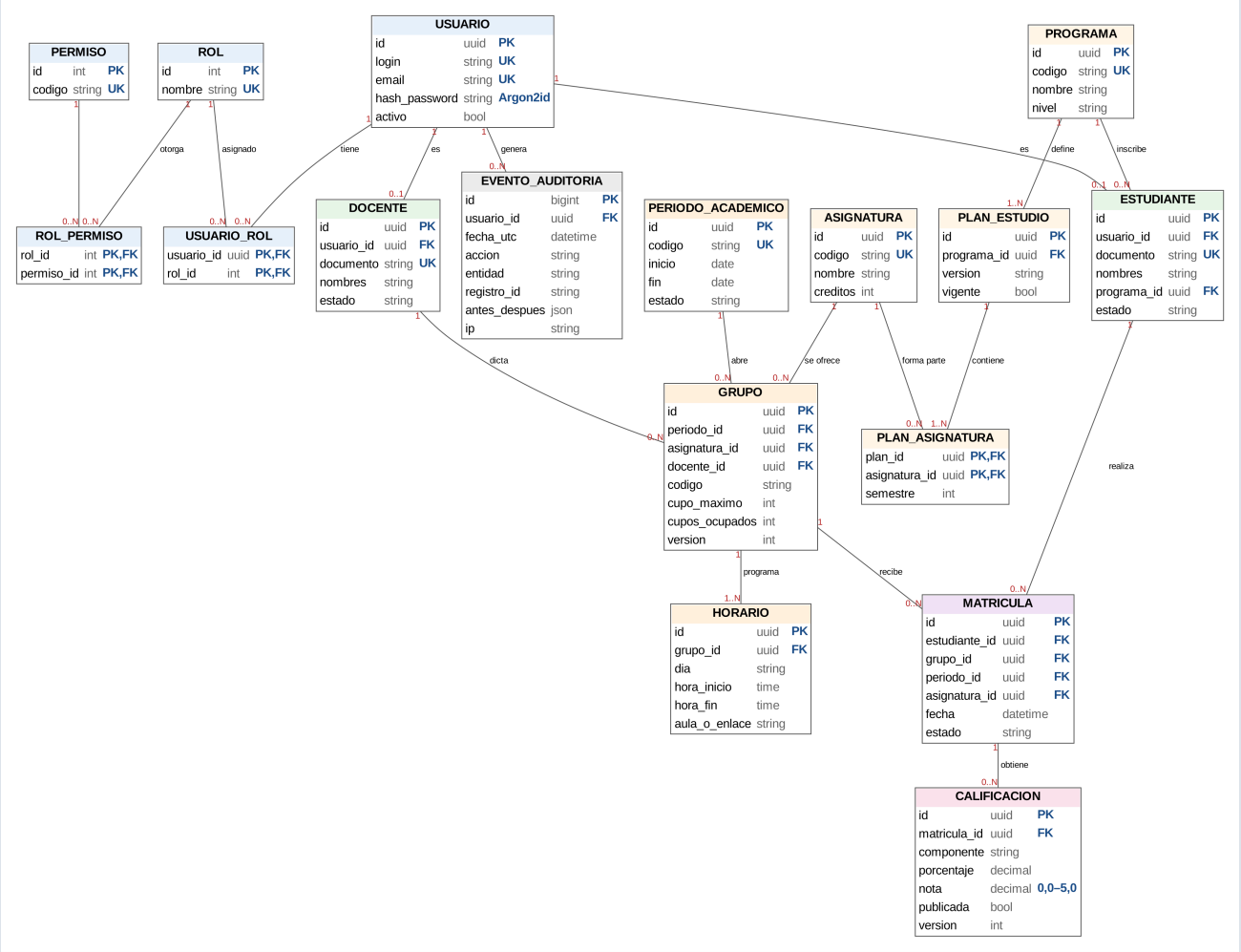


Figura 4. Modelo de datos conceptual. Las cifras en rojo en los extremos de cada relación indican la cardinalidad (1, 0..1, 0..N, 1..N). PK = clave primaria, FK = clave foránea, UK = valor único. Colores por módulo dueño de los datos.

Cardinalidades principales

Relación	Cardinalidad	Significado
Usuario — Rol (vía Usuario_Rol)	N : M	Un usuario puede tener varios roles (por ejemplo, docente y coordinador).
Rol — Permiso (vía Rol_Permiso)	N : M	Los permisos se agrupan por rol; permite ajustar sin cambiar código.
Usuario — Estudiante / Docente	1 : 0..1	Un usuario puede ser o no estudiante o docente; cada perfil tiene exactamente un usuario.

Relación	Cardinalidad	Significado
Programa — PlanEstudio	1 : 1..N	Un programa tiene una o más versiones de plan; solo una vigente.
PlanEstudio — Asignatura (vía Plan_Asignatura)	N : M	Una asignatura puede pertenecer a varios planes, con su semestre en cada uno.
Estudiante — Programa	N : 1	Cada estudiante está inscrito en un programa.
PeriodoAcadémico — Grupo	1 : 0..N	Cada grupo pertenece a un periodo.
Asignatura — Grupo	1 : 0..N	Una asignatura se ofrece en varios grupos por periodo.
Docente — Grupo	1 : 0..N	Cada grupo tiene un docente asignado.
Grupo — Horario	1 : 1..N	Un grupo tiene una o más franjas con aula o enlace.
Estudiante — Matrícula — Grupo	1 : 0..N : 1	La matrícula asocia un estudiante con un grupo en un periodo.
Matrícula — Calificación	1 : 0..N	Una nota por componente evaluativo de la matrícula.
Usuario — EventoAuditoría	1 : 0..N	Cada evento identifica al usuario que realizó la acción.

Restricciones de integridad relevantes

- Índice único parcial en `matricula(estudiante_id, asignatura_id, periodo_id) WHERE estado = 'ACTIVA'`: impide duplicados incluso si fallara la validación de la aplicación (RN-01). Por eso Matrícula guarda `asignatura_id` y `periodo_id` redundantes respecto del grupo.
- Restricción `CHECK (cupos_ocupados <= cupo_maximo)` en Grupo (RN-02, RN-08).
- Restricción `CHECK (nota BETWEEN 0 AND 5)` en Calificación (RN-04).
- Columnas `version` en Grupo y Calificación para control de concurrencia optimista.
- EventoAuditoría: el usuario de base de datos de la aplicación solo tiene permiso `INSERT` y `SELECT` sobre esta tabla.

6.5 Vistas de secuencia

Se modelan los dos procesos críticos que menciona el enunciado. El de matrícula es el más exigente porque combina control de acceso, concurrencia sobre un recurso limitado (cupos) y auditoría.

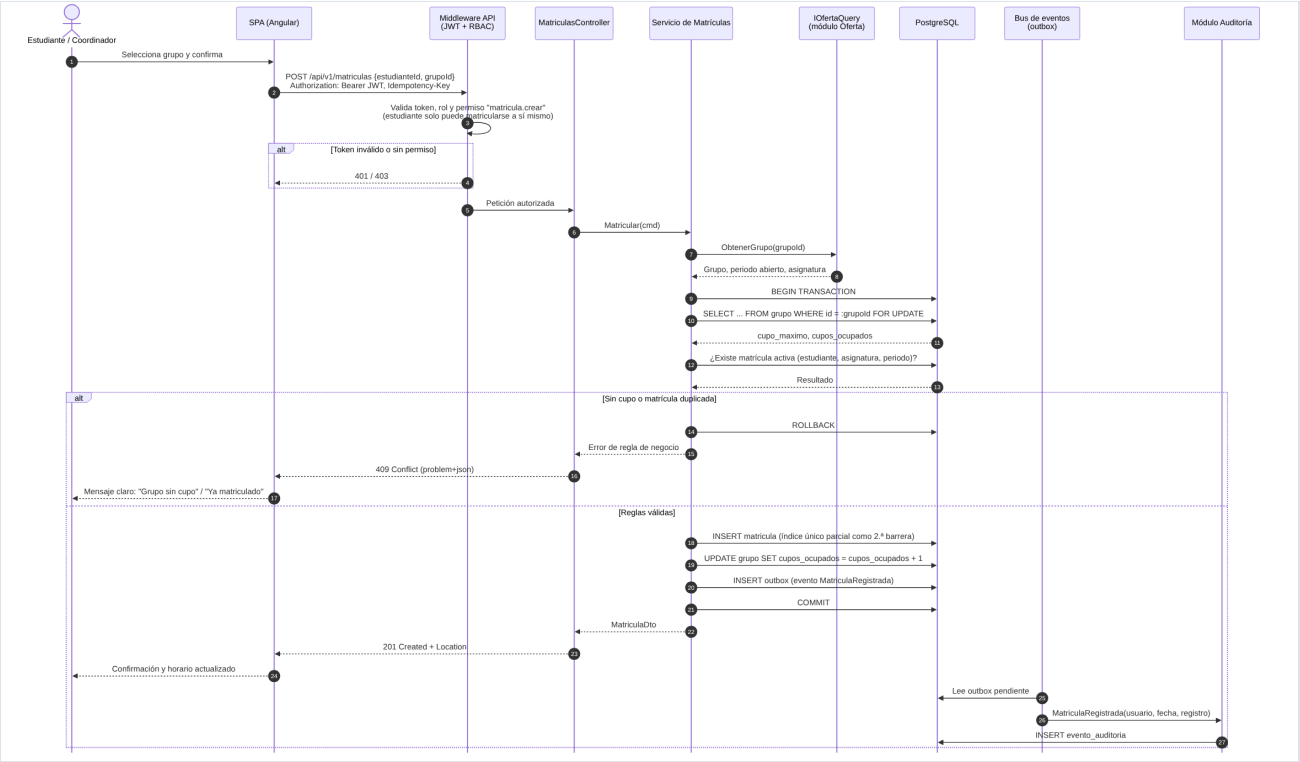


Figura 5. Diagrama de secuencia: matrícula de un estudiante en un grupo.

Lectura del diagrama. (1–4) El middleware rechaza peticiones sin token válido o sin permiso antes de tocar la lógica de negocio. (9–13) El servicio abre una transacción y bloquea la fila del grupo con `FOR UPDATE`: si 100 estudiantes intentan matricularse a la vez, las transacciones sobre ese grupo se serializan y el conteo de cupos nunca se lee desactualizado. (14–17) Si falla una regla, se revierte todo y el usuario recibe un mensaje claro. (18–21) Matrícula, incremento de cupos y evento *outbox* se confirman juntos, por lo que no puede existir una matrícula sin su registro de auditoría. (25–27) La auditoría se procesa de forma asíncrona y no penaliza el tiempo de respuesta.

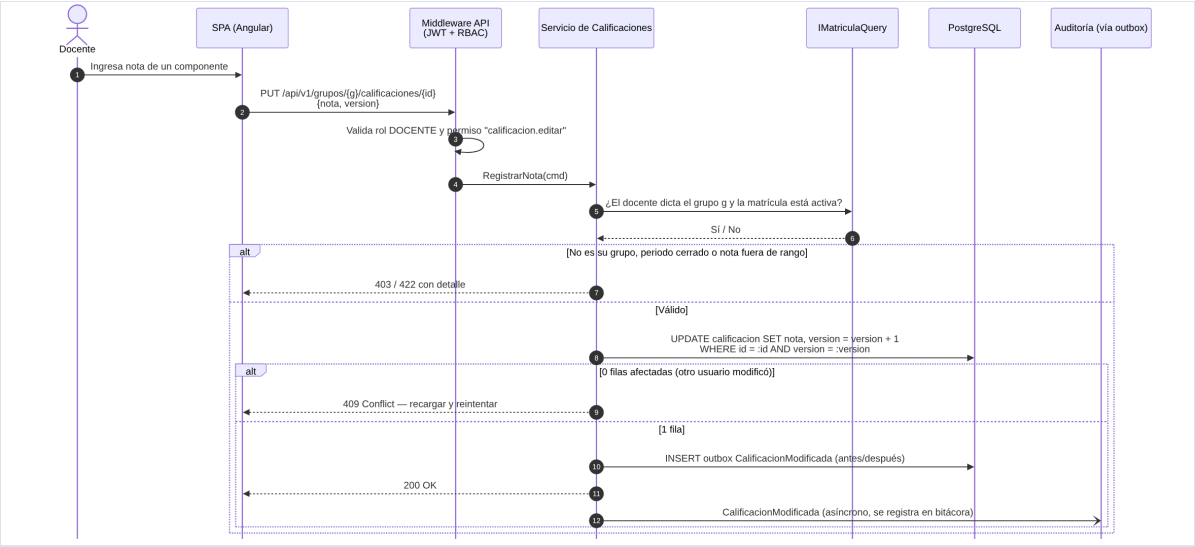


Figura 6. Diagrama de secuencia complementario: registro o actualización de una calificación con control de concurrencia optimista.

En las notas no se usa bloqueo pesimista porque los conflictos son raros (un grupo tiene un solo docente). Se usa control optimista: la actualización solo se aplica si la versión no cambió; si otra sesión modificó la nota, se responde 409 y la interfaz recarga.

7. Matriz de trazabilidad de requerimientos

Cada requerimiento se relaciona con los componentes que lo satisfacen, los modelos donde se representa, la decisión arquitectónica asociada (sección 8.1) y el método propuesto para verificar su cumplimiento.

ID	Componente(s)	Modelo(s)	ADR	Evidencia / método de verificación
RF-01	SPA (login), M1, middleware JWT	UC-01; Fig. 2	ADR-03	Pruebas de integración: login válido/ inválido, bloqueo tras 5 intentos, logout revoca refresh token.
RF-02	M1, middleware RBAC	UC-02; Fig. 2, 4 (Rol, Permiso)	ADR-03	Matriz rol × endpoint automatizada: cada combinación no permitida debe devolver 403.
RF-03	M2, SPA Administración	UC-03; Fig. 4 (Estudiante)	ADR-01	Pruebas de API del CRUD; verificar que desactivar conserva historial.
RF-04	M2, SPA Administración	UC-04; Fig. 4 (Docente)	ADR-01	Igual a RF-03; docente inactivo no asignable a grupo nuevo.
RF-05	M3	UC-05; Fig. 4 (Programa, Plan, Asignatura)	ADR-01	Pruebas de API; unicidad de código; asignatura en varios planes.
RF-06	M4	UC-06; Fig. 4 (Periodo, Grupo, Horario)	ADR-01	Pruebas unitarias de RN-08; pruebas de API de configuración.
RF-07	M5, M4 (`IOfertaQuery`), PostgreSQL	UC-07, UC-15; Fig. 5	ADR-02	Prueba de concurrencia: 100 solicitudes simultáneas a grupo de 30 cupos ⇒ exactamente 30 matrículas; prueba de idempotencia.
RF-08	M6, M5 (`IMatriculaQuery`)	UC-08, UC-09; Fig. 6	ADR-02	Pruebas unitarias de RN-03/04; prueba de edición concurrente ⇒ 409.
RF-09	SPA portal estudiante, M5, M6, M4	UC-10	ADR-03	Prueba E2E (Playwright); intento de leer datos de otro estudiante ⇒ 403/404.
RF-10	M7, Worker, almacenamiento de objetos	UC-11; Fig. 2, 3	ADR-05	Comparar totales del reporte con consultas SQL de control sobre datos de prueba.
RF-11	Controladores (paginación/filtros), índices BD	UC-12	—	Pruebas de API con filtros combinados; plan de ejecución SQL usa índices.
RF-12	M8, bus de eventos (outbox)	UC-13, UC-14; Fig. 5, 6; Fig. 4 (EventoAuditoría)	ADR-04	Por cada acción auditable, verificar evento con usuario, fecha, acción, registro; intento de UPDATE/DELETE en bitácora falla.
RNF-01	CDN/WAF, balanceador TLS, M1 (Argon2id), middleware	Fig. 3	ADR-03	SSL Labs ≥ A; OWASP ZAP sin hallazgos altos; revisión de que la columna de contraseña solo contiene hashes; lista de verificación OWASP ASVS nivel 2.

ID	Componente(s)	Modelo(s)	ADR	Evidencia / método de verificación
RNF-02	Caché Redis, índices, réplica para reportes, CDN	Fig. 3	ADR-05	Prueba de carga con k6: 300 usuarios virtuales, 30 min; p95 ≤ 3 s en las 4 consultas comunes.
RNF-03	2 instancias API, balanceador, réplica BD	Fig. 3	ADR-05	Monitoreo de disponibilidad externo; ensayo de falla: apagar una instancia y el primario sin pérdida de servicio prolongada.
RNF-04	API sin estado, autoescalado	Fig. 3	ADR-01, ADR-05	Prueba de carga al doble de usuarios con 4 instancias; verificar escalado sin cambios de código.
RNF-05	Módulos M1–M8, esquemas por módulo	Fig. 2	ADR-01	Pruebas de arquitectura en CI (dependencias prohibidas fallan el build); cobertura de dominio ≥ 70 %.
RNF-06	SPA Angular	—	ADR-06	axe-core / Lighthouse ≥ 90 en accesibilidad; recorrido manual solo con teclado.
RNF-07	M5, M6, PostgreSQL (transacciones, índices, CHECK)	Fig. 4, 5, 6	ADR-02	Las mismas pruebas de concurrencia de RF-07 y RF-08; inyección de falla a mitad de transacción ⇒ sin datos parciales.
RNF-08	Controladores REST, OpenAPI	Sección 5.5	ADR-06	Contrato OpenAPI publicado y validado en CI; pruebas de contrato.
RNF-09	PostgreSQL WAL, almacenamiento de objetos	Fig. 3	ADR-05	Simulacro de restauración por periodo: medir RPO y RTO reales y registrar el acta.
RNF-10	Este documento, ADR, repositorio	Sección 7	Todos	Revisión de que cada fila de esta matriz tiene componente y verificación.
RNF-11	Middleware RBAC, verificación de propiedad, logs sin datos personales	Fig. 2	ADR-03	Pruebas negativas de acceso cruzado; revisión de logs; inventario de datos personales.
RNF-12	SPA responsive, CDN	Fig. 3	ADR-06	Pruebas E2E en Chrome, Edge, Firefox y Safari (WebKit) a 360, 768 y 1920 px.

Cobertura: 12 de 12 RF y 12 de 12 RNF tienen al menos un componente y un método de verificación. Ningún componente quedó sin requerimiento asociado (Redis y el worker se justifican por RNF-02 y RF-10/RF-12).

8. Decisiones arquitectónicas, atributos de calidad y análisis de riesgos

8.1 Registro de decisiones arquitectónicas (ADR)

ADR-01 · Monolito modular en lugar de microservicios

Contexto: un solo dominio académico, 8 módulos, equipo de 3 a 5 personas, 300 usuarios concurrentes y una operación crítica (matrícula) que necesita consistencia fuerte.

Decisión: una aplicación desplegable con módulos de fronteras explícitas, esquema de base de datos por módulo y comunicación por interfaces o eventos.

Consecuencias: (+) transacciones locales simples, operación y costo bajos, un solo pipeline de despliegue. (−) un error grave puede afectar todo el sistema; se escala la aplicación completa y no un módulo. **Mitigación:** pruebas de arquitectura para preservar fronteras y opción de extraer módulos (por ejemplo, Reportes) si la carga lo exige.

ADR-02 · Base de datos relacional con bloqueo pesimista para cupos y optimista para notas

Contexto: RNF-07 exige que no se sobrepasen cupos ni se dupliquen matrículas ante solicitudes simultáneas; las notas rara vez tienen conflictos.

Decisión: PostgreSQL; matrícula con `SELECT … FOR UPDATE` sobre el grupo dentro de una transacción e índice único parcial como segunda barrera; calificaciones con columna `version`.

Consecuencias: (+) corrección garantizada por la base de datos y no solo por la aplicación. (−) en la apertura de matrícula, los grupos más solicitados serializan sus transacciones y pueden formar colas. **Mitigación:** transacciones cortas (sin llamadas externas dentro); `lock_timeout` de 2 s; límite de solicitudes concurrentes por grupo para no agotar el pool de conexiones; si un estudiante matricula varios grupos, cada grupo se procesa en su propia transacción (éxito parcial informado por asignatura) o los bloqueos se toman en orden ascendente de id de grupo para evitar interbloqueos; prueba de carga específica del día de apertura. La actualización condicional `UPDATE … WHERE cupos_ocupados < cupo_maximo` es una alternativa equivalente.

ADR-03 · JWT de corta duración + refresh token rotativo, RBAC por permisos y Argon2id

Contexto: RNF-01, RNF-11 y RF-02; la API debe ser sin estado para escalar (RNF-04).

Decisión: token de acceso de 15 minutos con permisos; refresh token rotativo en cookie *HttpOnly*, revocable en base de datos; autorización por permisos (no por nombre de rol) más verificación de propiedad del recurso; contraseñas con Argon2id según la guía OWASP.

Consecuencias: (+) escalado horizontal sin sesión compartida; permisos ajustables sin cambiar código. (−) un cambio de rol tarda hasta 15 min en reflejarse en tokens ya emitidos. **Mitigación:** al quitar un rol se revocan los refresh tokens del usuario; la ventana de 15 min queda documentada y aceptada.

ADR-04 · Auditoría basada en eventos con patrón outbox

Contexto: RF-12 exige registrar eventos sin perder ninguno; registrar sincrónicamente en un servicio externo podría fallar o ser lento.

Decisión: el evento se escribe en una tabla *outbox* dentro de la misma transacción de negocio; el worker lo traslada a la bitácora inmutable.

Consecuencias: (+) no puede existir una matrícula o nota sin evento; no penaliza la latencia. (−) la bitácora puede tener segundos de retraso respecto de la operación. **Mitigación:** alerta si la outbox acumula eventos sin procesar por más de 5 minutos.

ADR-05 · Alta disponibilidad con redundancia activa y respaldos PITR

Contexto: RNF-03 (99,5 %) y RNF-09 (respaldo y restauración).

Decisión: dos instancias de API en zonas distintas tras un balanceador; PostgreSQL con réplica en espera; respaldo base diario + WAL continuo a almacenamiento de objetos en otra región.

Consecuencias: (+) tolera la caída de una instancia o una zona; permite restaurar a un punto en el tiempo (por ejemplo, antes de una carga masiva de notas errónea). (−) mayor costo que un servidor único. **Mitigación:** instancias pequeñas con autoescalado y servicios administrados.

ADR-06 · SPA Angular + API REST con contrato OpenAPI

Contexto: RNF-06, RNF-08 y RNF-12: interfaz accesible y adaptable, e integración futura con otras plataformas.

Decisión: separar cliente (SPA) y servidor (API REST versionada). La misma API que usa la SPA queda disponible para integraciones.

Consecuencias: (+) un solo contrato para la interfaz y las integraciones; la SPA se sirve desde CDN. (−) la SPA requiere cuidar accesibilidad de forma explícita (gestión del foco, anuncios a lectores de pantalla). **Mitigación:** pruebas automáticas de accesibilidad en CI.

8.2 Evaluación frente a los atributos de calidad

Atributo	Grado de cumplimiento esperado	Argumento
Seguridad	Alto	Defensa en capas: WAF, TLS, red privada, autenticación, autorización por permiso y por propiedad, hash de contraseñas, auditoría.
Rendimiento	Alto con margen	Estimación: 300 usuarios con una petición cada ~10 s ≈ 30 peticiones/s; con pico ×3 ≈ 90–100 peticiones/s, carga moderada para dos instancias con caché e índices. Debe confirmarse con la prueba de carga.
Disponibilidad	Adecuado	Sin punto único de falla en la ruta principal salvo el primario de BD durante la conmutación (minutos), compatible con el margen mensual de ~3,6 h.
Escalabilidad	Adecuado	Escalado horizontal de la API; el límite real es la base de datos de escritura (sección 8.5).
Mantenibilidad	Alto si se respeta la disciplina modular	Depende de que las pruebas de arquitectura impidan dependencias indebidas.
Usabilidad / accesibilidad	Medio a alto	La arquitectura lo permite; el cumplimiento real depende de la implementación de la interfaz.

8.3 Riesgos

ID	Riesgo	Prob.	Imp.	Mitigación / respuesta
R1	Pico de solicitudes al abrir la matrícula satura grupos populares y la base de datos.	Alta	Alto	Prueba de carga del escenario de apertura; apertura escalonada por semestre o programa; tiempo máximo de bloqueo; mensajes claros y reintento controlado.
R2	El monolito se degrada en un «gran bola de lodo» por dependencias entre módulos.	Media	Alto	Pruebas de arquitectura obligatorias en CI; revisión de código; un esquema por módulo con permisos de BD separados.
R3	Robo de credenciales o de tokens (phishing, XSS).	Media	Alto	Tokens de corta vida, refresh en cookie *HttpOnly*, CSP, bloqueo por intentos; recomendación de segundo factor para administradores.
R4	Escalamiento de privilegios: un estudiante accede a datos de otro (IDOR).	Media	Alto	Verificación de propiedad en cada caso de uso; pruebas negativas automatizadas; identificadores UUID no secuenciales.
R5	Respaldos que no restauran (nunca probados).	Media	Muy alto	Simulacro de restauración cada periodo con acta; alerta si falla el archivado de WAL.
R6	Migración de datos desde archivos actuales con duplicados e inconsistencias.	Alta	Alto	Proyecto de migración separado: limpieza, deduplicación por documento, validación por muestreo y carga por etapas.
R7	Modificación indebida de notas por un usuario con permisos legítimos.	Baja	Alto	Bitácora inmutable con valor anterior y nuevo; motivo obligatorio tras publicación; reportes de cambios para el coordinador.
R8	Dependencia del proveedor de nube (lock-in) y costos crecientes.	Media	Medio	Contenedores Docker y PostgreSQL estándar, portables entre proveedores; presupuesto con alertas.

ID	Riesgo	Prob.	Imp.	Mitigación / respuesta
R9	Incumplimiento de la Ley 1581 por exposición de datos en logs o reportes exportados.	Media	Alto	Enmascaramiento en logs; reportes exportados con caducidad de enlace; clasificación de datos personales.
R10	Retraso en la bitácora (outbox) oculta acciones durante una investigación.	Baja	Medio	Alerta por eventos pendientes > 5 min; los eventos ya están persistidos en la outbox aunque el worker falle.
R11	Interbloqueos al matricular varios grupos a la vez y agotamiento del pool de conexiones por transacciones esperando bloqueos (hallazgo de la revisión con IA, sección 9.3).	Media	Alto	Una transacción por grupo o bloqueo en orden determinista; `lock_timeout` corto; límite de concurrencia por grupo; pool separado para lecturas.

8.4 Puntos únicos de falla y problemas de seguridad

- **Base de datos primaria:** es el punto más sensible. La réplica en espera reduce el impacto, pero la conmutación toma de uno a dos minutos y puede perder las últimas transacciones si la replicación es asíncrona. Se recomienda replicación síncrona (más latencia de escritura, sin pérdida de datos confirmados), a validar con la prueba de carga.
- **Balanceador y CDN:** se usan como servicios administrados redundantes del proveedor; si se autogestionaran, serían un punto único de falla.
- **Gestor de secretos y proveedor de identidad:** si M1 falla, nadie puede iniciar sesión; las sesiones vigentes siguen funcionando hasta que vence el token.
- **Redis:** no es punto único de falla: el sistema está diseñado para funcionar sin caché, con más latencia.
- **Amenazas principales (enfoque STRIDE):** suplantación (credenciales robadas), manipulación (cambio de notas), repudio (negar un cambio, mitigado por auditoría), divulgación (IDOR, logs), denegación de servicio (picos de matrícula, ataques volumétricos mitigados por WAF y rate limiting) y elevación de privilegios (asignación indebida de roles, auditada en RF-12).

8.5 Límites de escalabilidad

La API escala horizontalmente casi sin límite práctico para este contexto. El límite real está en la **base de datos de escritura** (un solo primario) y, dentro de ella, en la **contención de bloqueos** sobre los grupos más demandados durante la matrícula. Si la institución creciera mucho más allá del escenario de referencia, las siguientes medidas se aplicarían en este orden: (1) más réplicas de lectura para consultas y reportes; (2) cola de solicitudes de matrícula por grupo; (3) particionar las tablas de matrícula y calificación por periodo; (4) extraer el módulo de Matrículas como servicio independiente con su propia base de datos. La arquitectura modular permite dar estos pasos sin rediseñar todo el sistema, como pide RNF-04. Las señales concretas para extraer un módulo serían: picos sostenidos de inscripción muy superiores al escenario de referencia que saturen la base de datos, un equipo que crezca a varias células independientes (más de 15 desarrolladores), o un módulo con requisitos regulatorios o de despliegue propios (por ejemplo, integración con pagos).

8.6 Decisiones que requieren validación adicional

1. RPO ≤ 15 min y RTO ≤ 4 h: deben confirmarse con la institución, junto con el modo de replicación (síncrona o asíncrona).
2. Volumen real de usuarios y comportamiento del día de apertura de matrícula: validar con datos históricos antes de dimensionar.
3. Si existe un proveedor de identidad institucional (Microsoft Entra ID, Google Workspace): en ese caso, M1 delegaría la autenticación vía OIDC.
4. Escala de calificación y reglas de componentes por programa (S-02).
5. Política de retención de la bitácora de auditoría y de los respaldos frente a la normativa de datos personales.
6. Proveedor de nube y costo mensual estimado.
7. Reglas de matrícula no definidas en el enunciado: reserva temporal de cupos, turnos escalonados (por semestre o promedio) y prerrequisitos.
8. Copias de respaldo inmutables (WORM) y si el objetivo de RTO debe bajar a 30 min, como sugirió la revisión con IA.

9. Evidencias del uso de Google AI Studio

9.1 Metodología de uso crítico

1. **Una conversación, un prompt por etapa:** los cinco prompts se ejecutaron en una misma conversación de Google AI Studio («Análisis de Sistema de Gestión», 8 de noviembre de 2026, 12:08 a 12:35 p. m.), con la opción *Grounding with Google Search* activa. Así cada respuesta podía revisarse antes de pasar a la siguiente etapa.
2. **Sin datos reales:** no se incluyeron nombres, documentos ni credenciales reales (R-05).
3. **Verificación:** cada respuesta se contrastó contra (a) los requerimientos del enunciado, (b) la coherencia con los demás modelos y (c) bibliografía de referencia (Bass et al., 2021; Richards & Ford, 2020; OWASP Foundation, 2021).
4. **Decisión explícita:** cada sugerencia se clasificó como Aceptada, Modificada o Descartada, con su justificación (sección 9.3).

> **Limitación detectada en el propio proceso.** En los prompts 1 y 5 quedó sin reemplazar el marcador `[pegar texto…]`, por lo que la IA no recibió el texto literal de los requerimientos ni el resumen de la propuesta. En el prompt 1 la herramienta no inventó requerimientos: pidió el texto y propuso la estructura del análisis (Evidencia 2). En el prompt 5 lo advirtió («Nota previa») y revisó los artefactos generados en la misma conversación. Por eso las respuestas se trataron como insumo de contraste y no como fuente de requerimientos: los RF/RNF del informe provienen únicamente del enunciado.

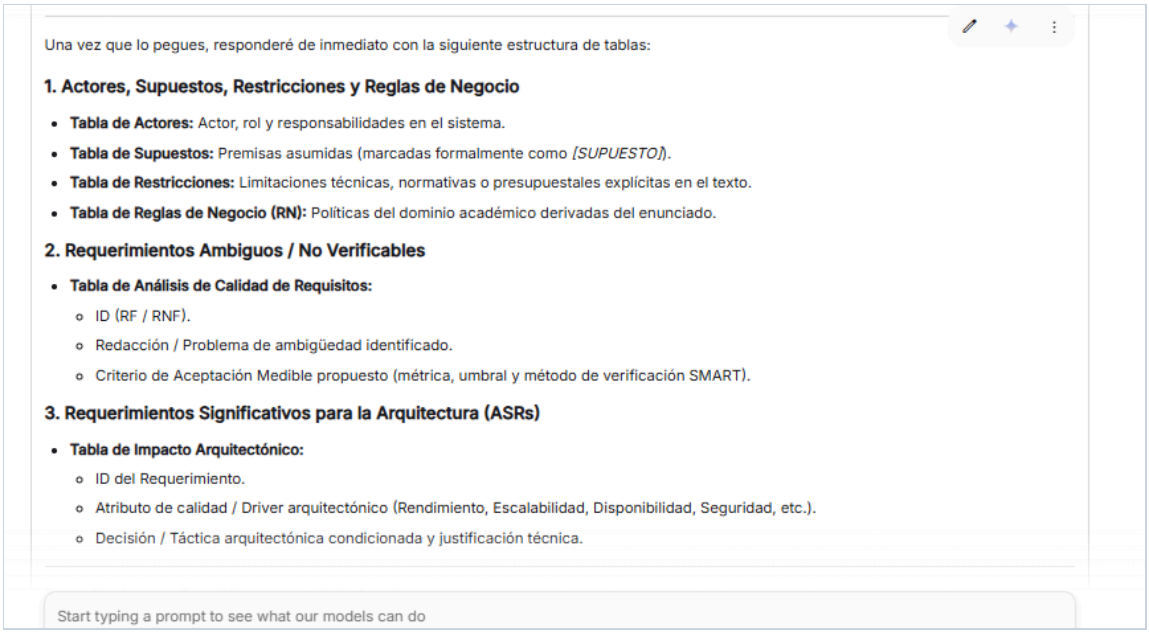
9.2 Prompts y respuestas (capturas)

Prompt 1 — Análisis de requerimientos

```
Actúa como arquitecto de software. Te comparto el caso de un Sistema de Gestión Académica
(SGA) para una institución de educación superior y sus requerimientos RF-01 a RF-12 y RNF-01
a RNF-12 [...]. 1) Identifica actores, supuestos, restricciones y reglas de negocio. 2)
Señala qué requerimientos son ambiguos o no verificables y propón un criterio de aceptación
medible para cada uno. 3) Indica cuáles requerimientos condicionan más la arquitectura y por
qué. Responde en tablas y no inventes requerimientos…
```

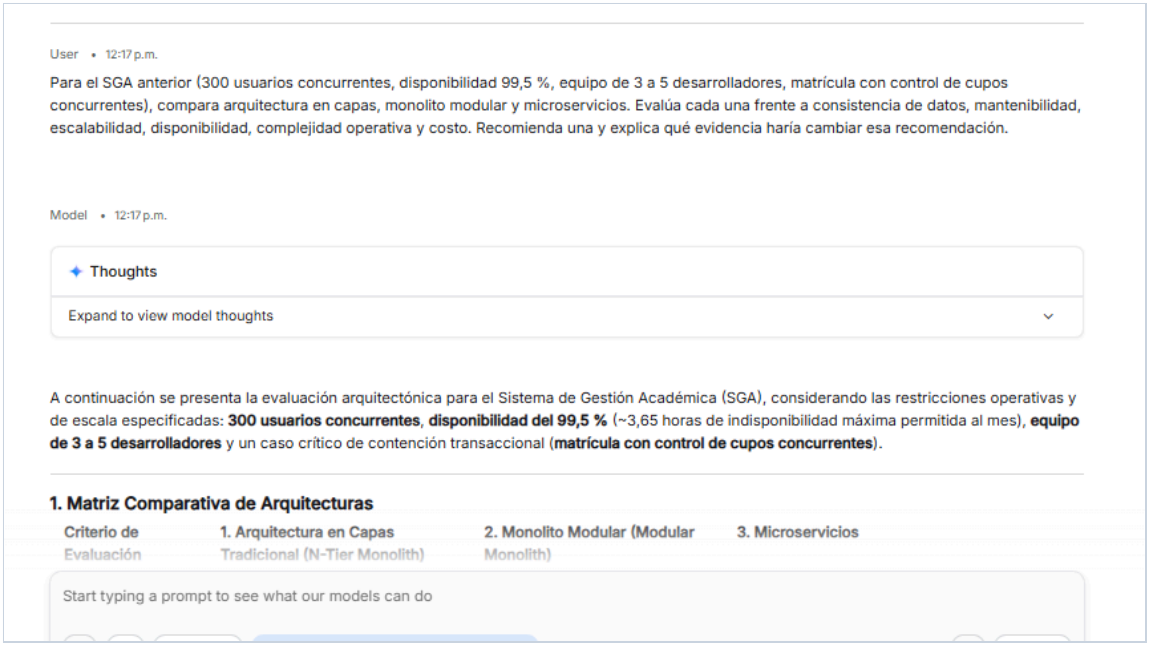


Evidencia 1. Prompt 1 enviado en Google AI Studio.

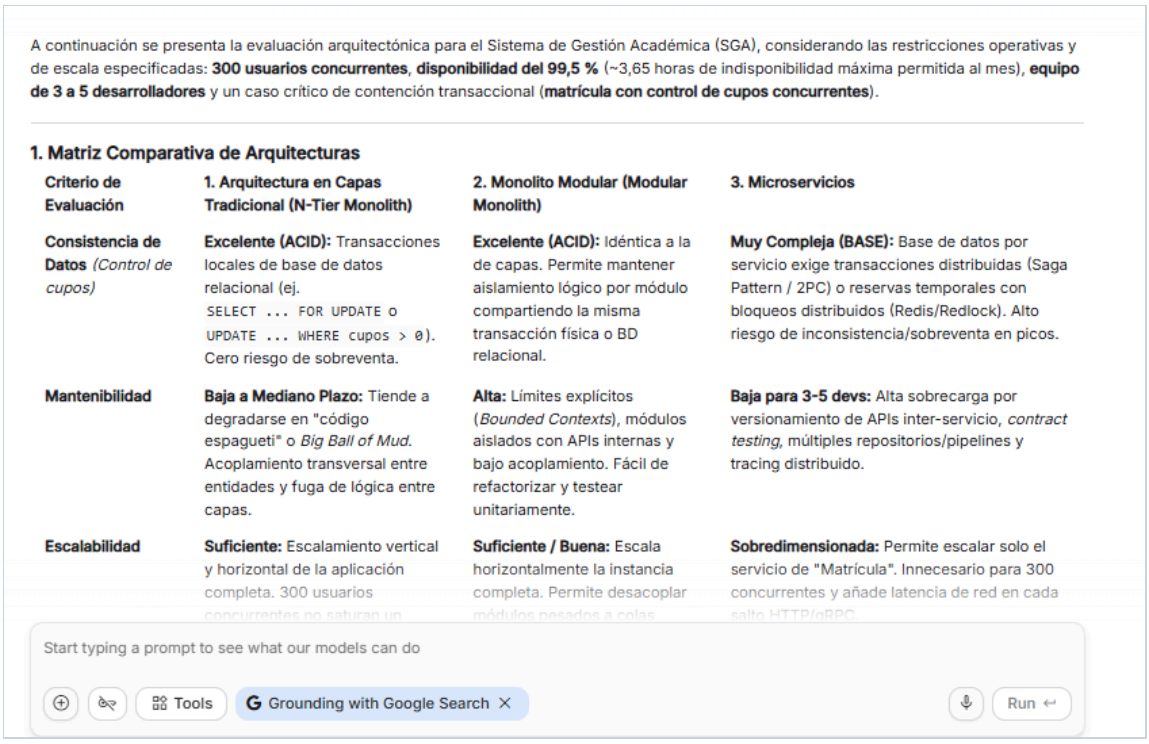


Evidencia 2. Respuesta al Prompt 1: la IA solicita el texto del enunciado y propone la estructura del análisis.

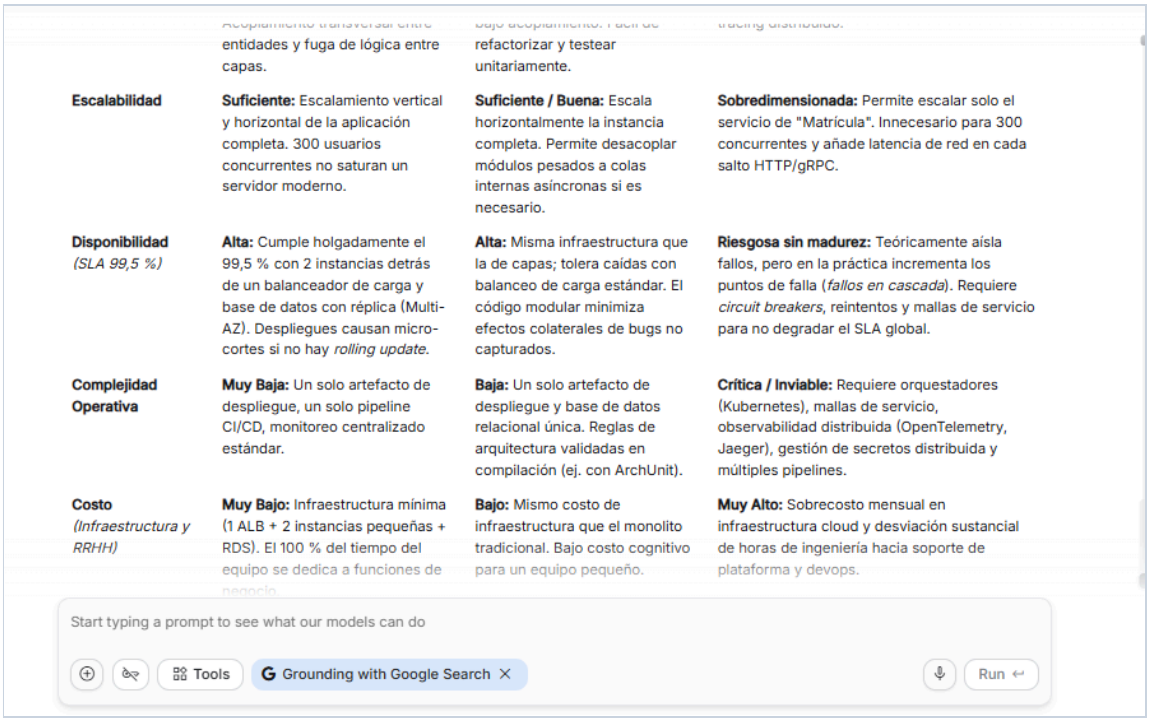
Prompt 2 — Selección del estilo arquitectónico



Evidencia 3. Prompt 2 y comienzo de la respuesta.

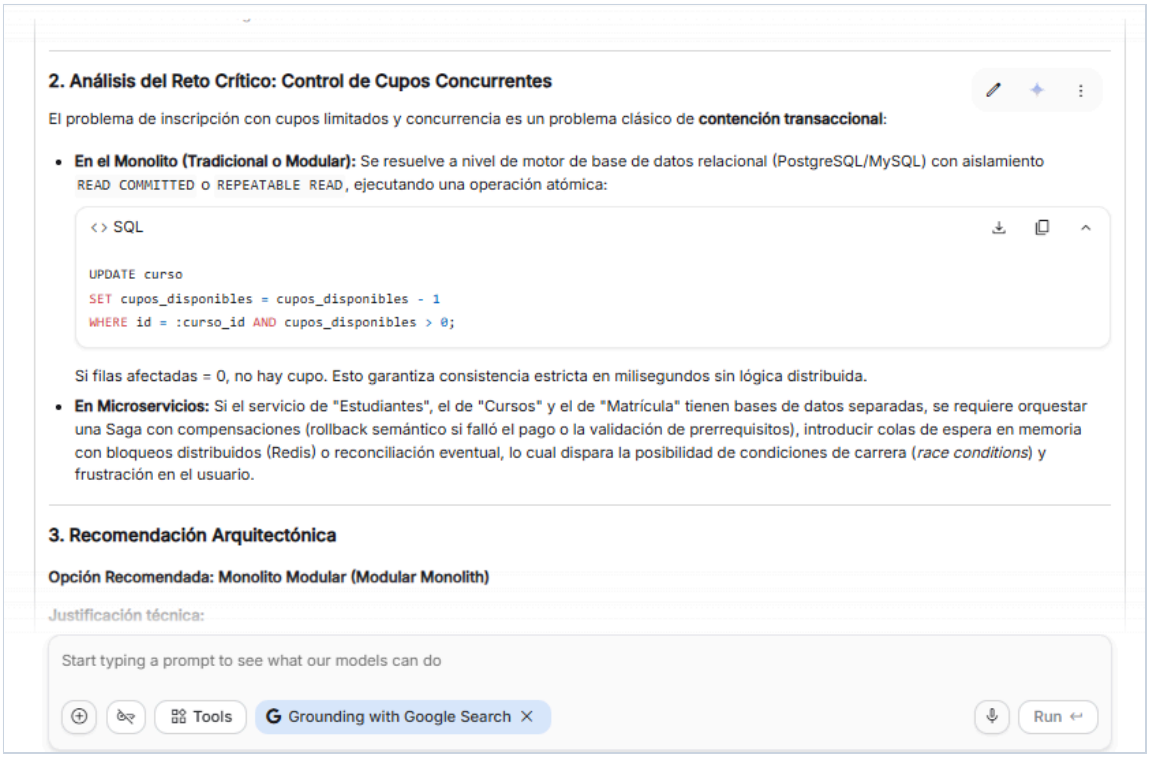


Evidencia 4. Matriz comparativa: consistencia, mantenibilidad y escalabilidad.



	entidades y fuga de lógica entre capas.	refactorizar y testear unitariamente.	
Escalabilidad	**Suficiente:** Escalamiento vertical y horizontal de la aplicación completa. 300 usuarios concurrentes no saturan un servidor moderno.	**Suficiente / Buena:** Escala horizontalmente la instancia completa. Permite desacoplar módulos pesados a colas internas asíncronas si es necesario.	**Sobredimensionada:** Permite escalar solo el servicio de "Matrícula". Innecesario para 300 concurrentes y añade latencia de red en cada salto HTTP/gRPC.
Disponibilidad *(SLA 99,5 %)*	**Alta:** Cumple holgadamente el 99,5 % con 2 instancias detrás de un balanceador de carga y base de datos con réplica (Multi-AZ). Despliegues causan micro-cortes si no hay *rolling update*.	**Alta:** Misma infraestructura que la de capas; tolera caídas con balanceo de carga estándar. El código modular minimiza efectos colaterales de bugs no capturados.	**Riesgosa sin madurez:** Teóricamente aísla fallos, pero en la práctica incrementa los puntos de falla (*fallos en cascada*). Requiere *circuit breakers*, reintentos y mallas de servicio para no degradar el SLA global.
Complejidad Operativa	**Muy Baja:** Un solo artefacto de despliegue, un solo pipeline CI/CD, monitoreo centralizado estándar.	**Baja:** Un solo artefacto de despliegue y base de datos relacional única. Reglas de arquitectura validadas en compilación (ej. con ArchUnit).	**Crítica / Inviable:** Requiere orquestadores (Kubernetes), mallas de servicio, observabilidad distribuida (OpenTelemetry, Jaeger), gestión de secretos distribuida y múltiples pipelines.
Costo *(Infraestructura y RRHH)*	**Muy Bajo:** Infraestructura mínima (1 ALB + 2 instancias pequeñas + RDS). El 100 % del tiempo del equipo se dedica a funciones de negocio.	**Bajo:** Mismo costo de infraestructura que el monolito tradicional. Bajo costo cognitivo para un equipo pequeño.	**Muy Alto:** Sobrecosto mensual en infraestructura cloud y desviación sustancial de horas de ingeniería hacia soporte de plataforma y devops.

Evidencia 5. Matriz comparativa: disponibilidad, complejidad operativa y costo.



2. Análisis del Reto Crítico: Control de Cupos Concurrentes

El problema de inscripción con cupos limitados y concurrencia es un problema clásico de **contención transaccional**:

- **En el Monolito (Tradicional o Modular):** Se resuelve a nivel de motor de base de datos relacional (PostgreSQL/MySQL) con aislamiento `READ COMMITTED` o `REPEATABLE READ`, ejecutando una operación atómica:

```sql
UPDATE curso
SET cupos_disponibles = cupos_disponibles - 1
WHERE id = :curso_id AND cupos_disponibles > 0;
```

  Si filas afectadas = 0, no hay cupo. Esto garantiza consistencia estricta en milisegundos sin lógica distribuida.

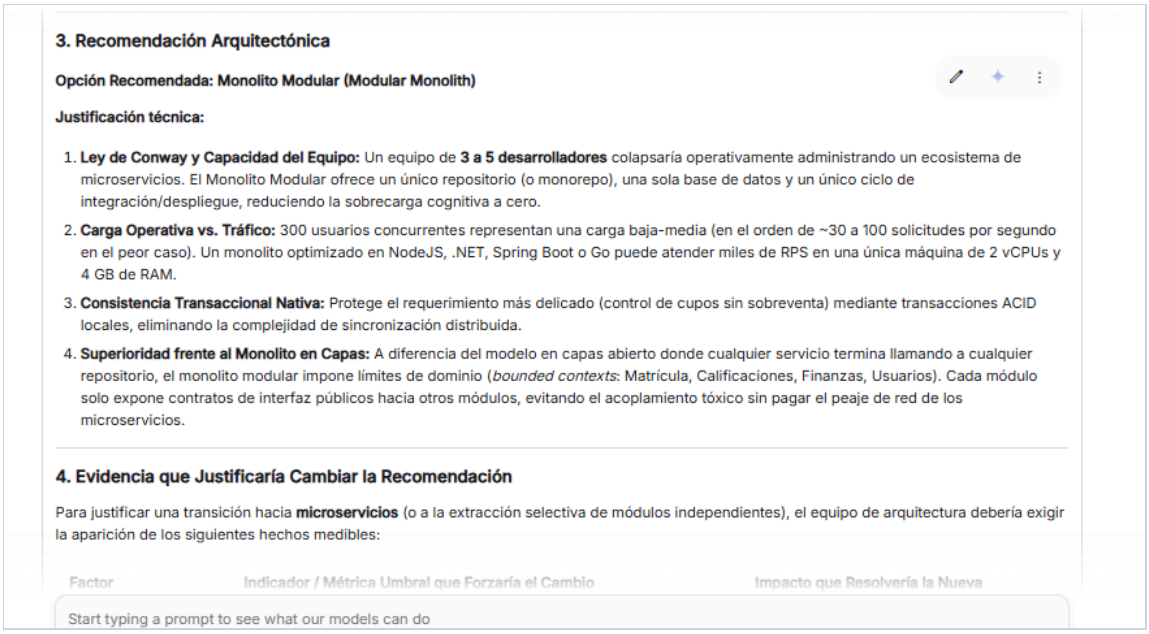
- **En Microservicios:** Si el servicio de "Estudiantes", el de "Cursos" y el de "Matrícula" tienen bases de datos separadas, se requiere orquestar una Saga con compensaciones (rollback semántico si falló el pago o la validación de prerrequisitos), introducir colas de espera en memoria con bloqueos distribuidos (Redis) o reconciliación eventual, lo cual dispara la posibilidad de condiciones de carrera (*race conditions*) y frustración en el usuario.

3. Recomendación Arquitectónica

Opción Recomendada: Monolito Modular (Modular Monolith)

Justificación técnica:

Evidencia 6. Análisis del control de cupos concurrentes (actualización condicional en SQL).



3. Recomendación Arquitectónica

Opción Recomendada: Monolito Modular (Modular Monolith)

Justificación técnica:

1. **Ley de Conway y Capacidad del Equipo:** Un equipo de **3 a 5 desarrolladores** colapsaría operativamente administrando un ecosistema de microservicios. El Monolito Modular ofrece un único repositorio (o monorepo), una sola base de datos y un único ciclo de integración/despliegue, reduciendo la sobrecarga cognitiva a cero.
2. **Carga Operativa vs. Tráfico:** 300 usuarios concurrentes representan una carga baja-media (en el orden de ~30 a 100 solicitudes por segundo en el peor caso). Un monolito optimizado en NodeJS, .NET, Spring Boot o Go puede atender miles de RPS en una única máquina de 2 vCPUs y 4 GB de RAM.
3. **Consistencia Transaccional Nativa:** Protege el requerimiento más delicado (control de cupos sin sobreventa) mediante transacciones ACID locales, eliminando la complejidad de sincronización distribuida.
4. **Superioridad frente al Monolito en Capas:** A diferencia del modelo en capas abierto donde cualquier servicio termina llamando a cualquier repositorio, el monolito modular impone límites de dominio (*bounded contexts*: Matrícula, Calificaciones, Finanzas, Usuarios). Cada módulo solo expone contratos de interfaz públicos hacia otros módulos, evitando el acoplamiento tóxico sin pagar el peaje de red de los microservicios.

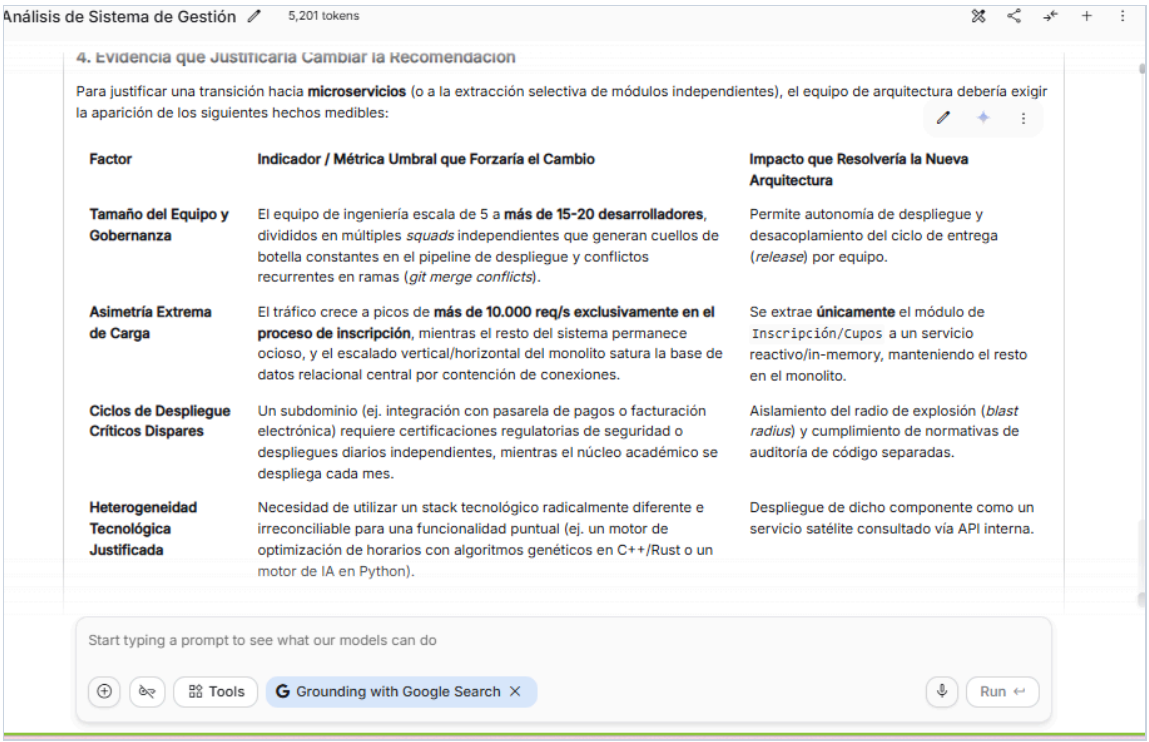
4. Evidencia que Justificaría Cambiar la Recomendación

Para justificar una transición hacia **microservicios** (o a la extracción selectiva de módulos independientes), el equipo de arquitectura debería exigir la aparición de los siguientes hechos medibles:

Factor | Indicador / Métrica Umbral que Forzaría el Cambio | Impacto que Resolvería la Nueva

Start typing a prompt to see what our models can do

Evidencia 7. Recomendación: monolito modular y su justificación técnica.



Análisis de Sistema de Gestión 5,201 tokens

4. Evidencia que Justificaría Cambiar la Recomendación

Para justificar una transición hacia **microservicios** (o a la extracción selectiva de módulos independientes), el equipo de arquitectura debería exigir la aparición de los siguientes hechos medibles:

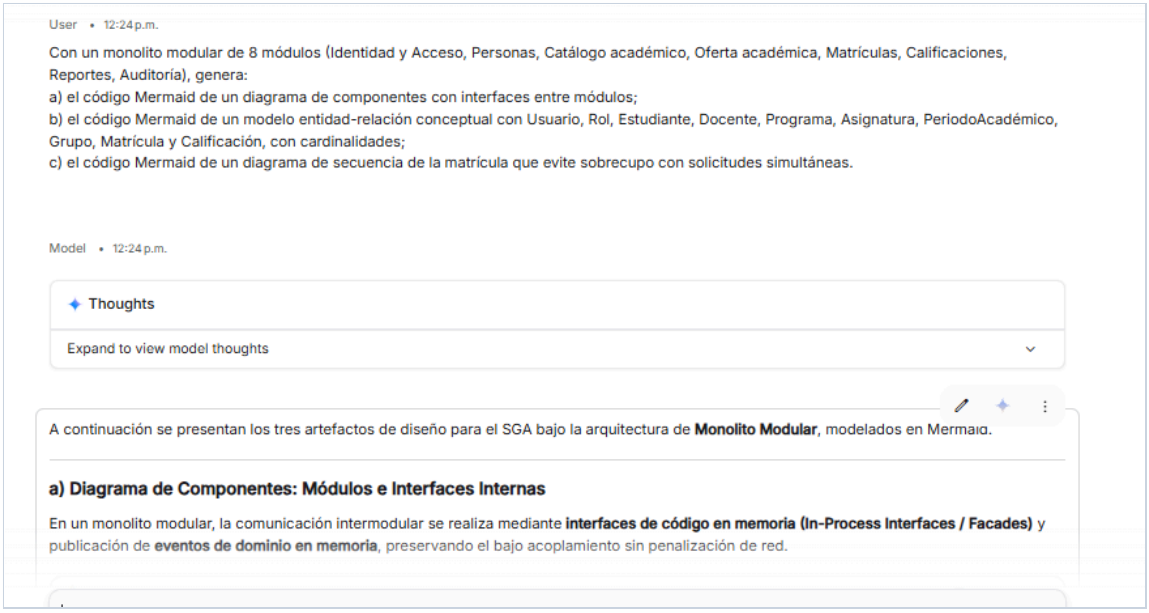
Factor	Indicador / Métrica Umbral que Forzaría el Cambio	Impacto que Resolvería la Nueva Arquitectura
Tamaño del Equipo y Gobernanza	El equipo de ingeniería escala de 5 a **más de 15-20 desarrolladores**, divididos en múltiples *squads* independientes que generan cuellos de botella constantes en el pipeline de despliegue y conflictos recurrentes en ramas (*git merge conflicts*).	Permite autonomía de despliegue y desacoplamiento del ciclo de entrega (*release*) por equipo.
Asimetría Extrema de Carga	El tráfico crece a picos de **más de 10.000 req/s exclusivamente en el proceso de inscripción**, mientras el resto del sistema permanece ocioso, y el escalado vertical/horizontal del monolito satura la base de datos relacional central por contención de conexiones.	Se extrae **únicamente** el módulo de `Inscripción/Cupos` a un servicio reactivo/in-memory, manteniendo el resto en el monolito.
Ciclos de Despliegue Críticos Dispares	Un subdominio (ej. integración con pasarela de pagos o facturación electrónica) requiere certificaciones regulatorias de seguridad o despliegues diarios independientes, mientras el núcleo académico se despliega cada mes.	Aislamiento del radio de explosión (*blast radius*) y cumplimiento de normativas de auditoría de código separadas.
Heterogeneidad Tecnológica Justificada	Necesidad de utilizar un stack tecnológico radicalmente diferente e irreconciliable para una funcionalidad puntual (ej. un motor de optimización de horarios con algoritmos genéticos en C++/Rust o un motor de IA en Python).	Despliegue de dicho componente como un servicio satélite consultado vía API interna.

Start typing a prompt to see what our models can do

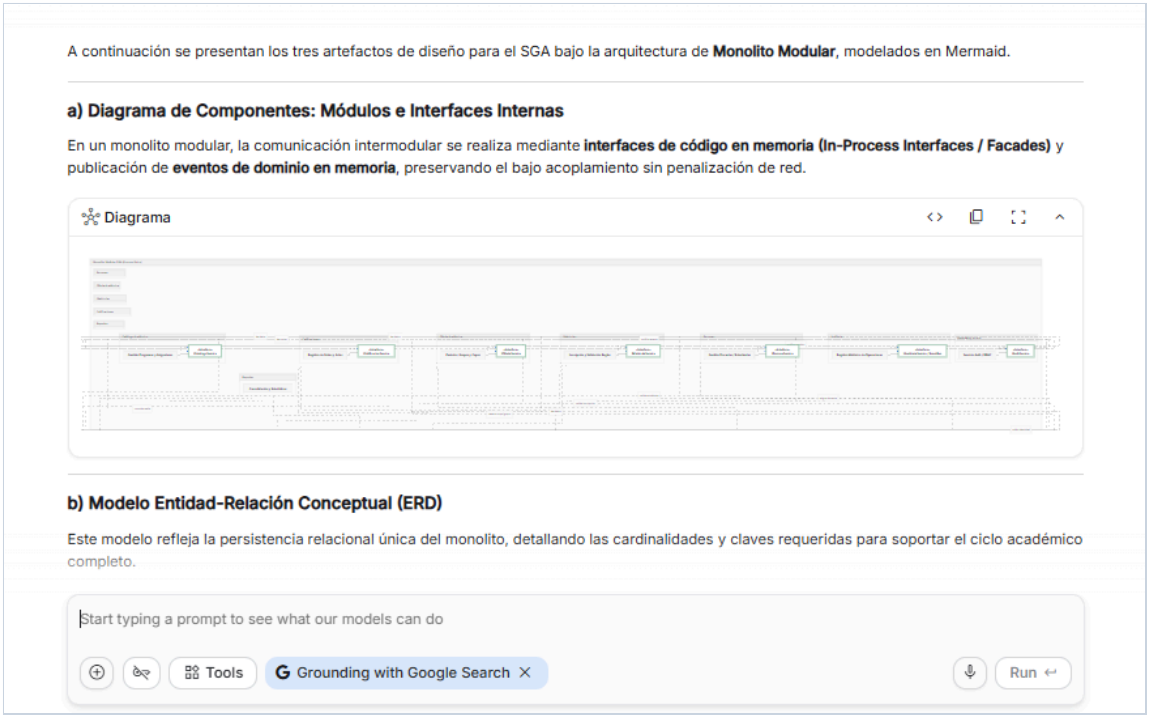
Tools | Grounding with Google Search | Run

Evidencia 8. Evidencia que justificaría migrar a microservicios.

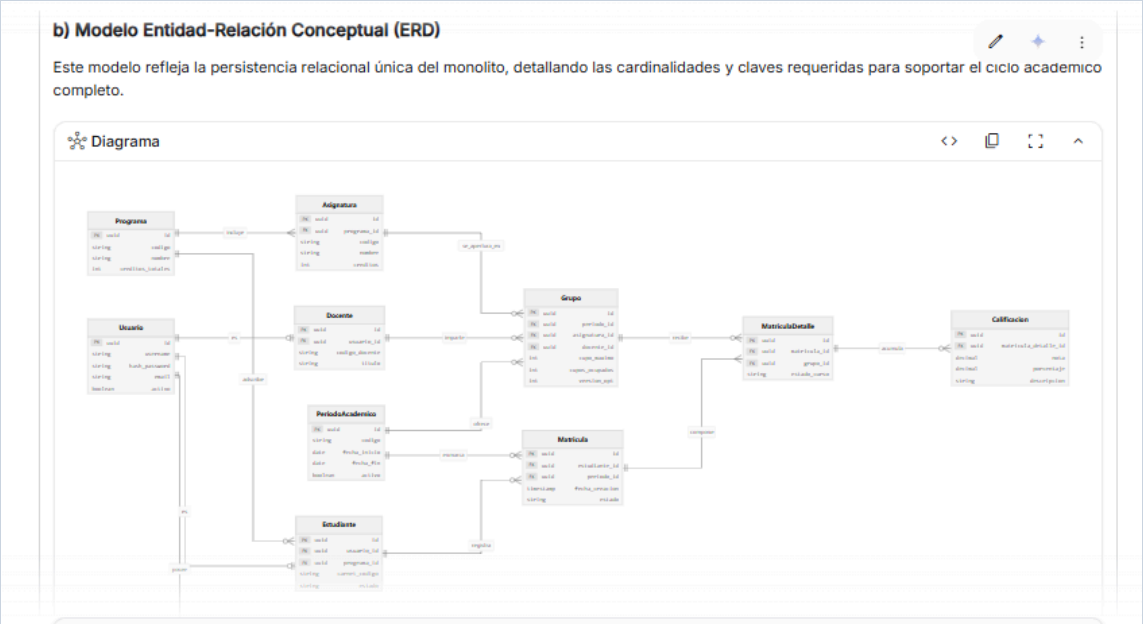
Prompt 3 — Modelos del sistema (Mermaid)



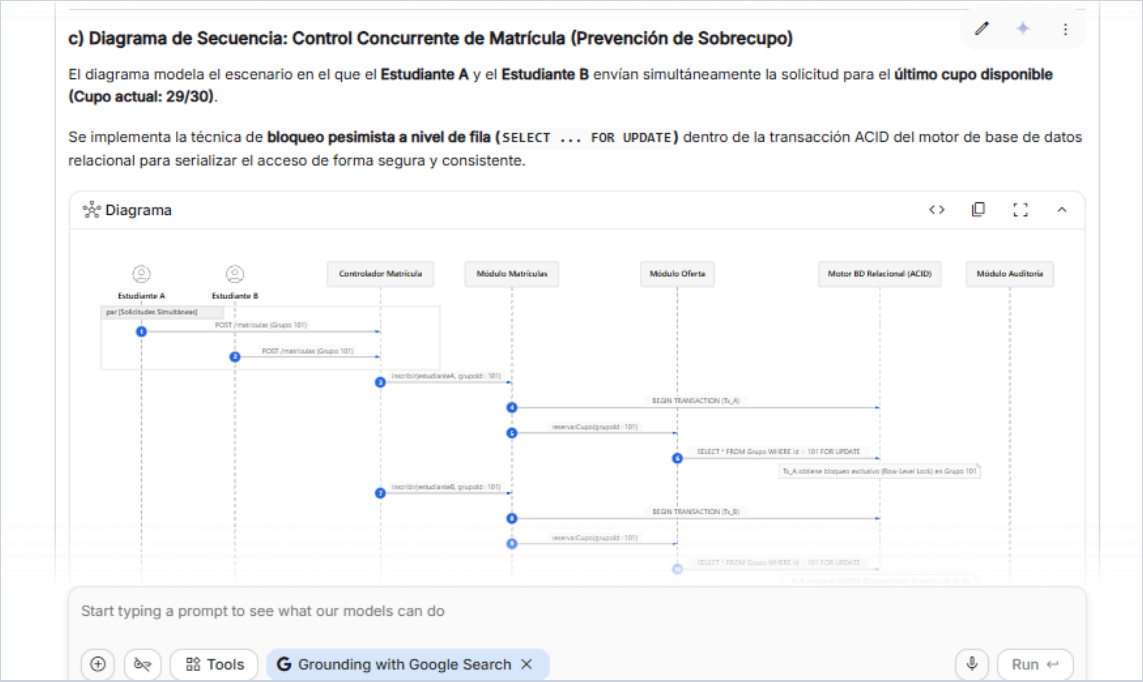
Evidencia 9. Prompt 3 y comienzo de la respuesta.



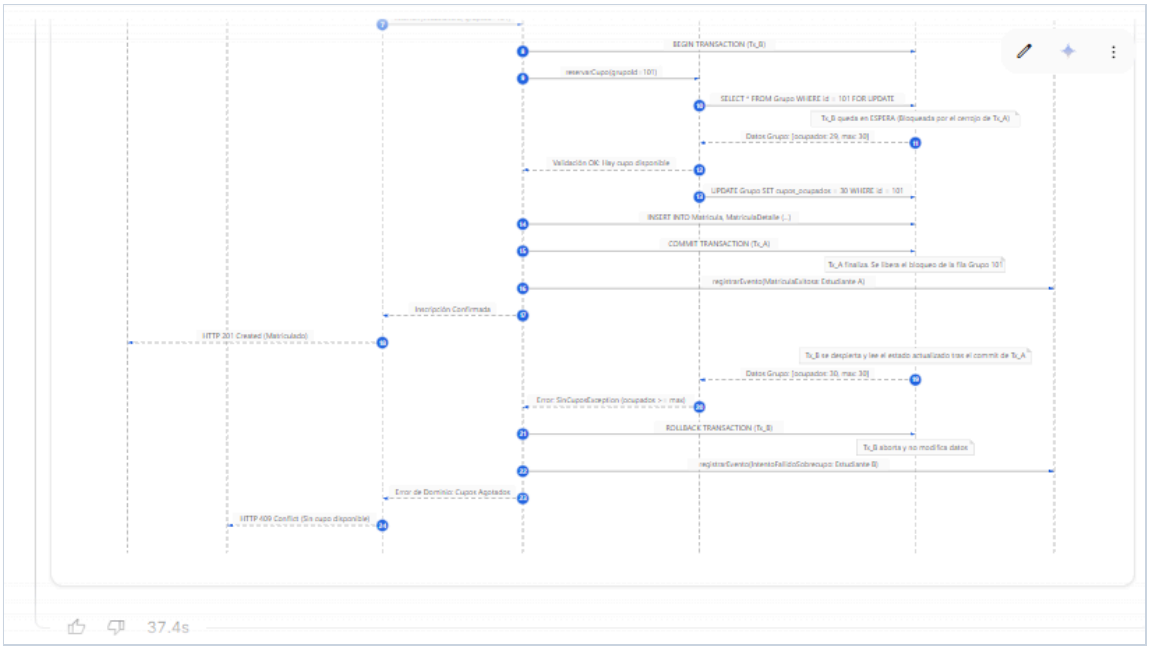
Evidencia 10. Diagrama de componentes generado (ilegible por su ancho).



Evidencia 11. Modelo entidad-relación generado (incluye la entidad MatriculaDetalle).

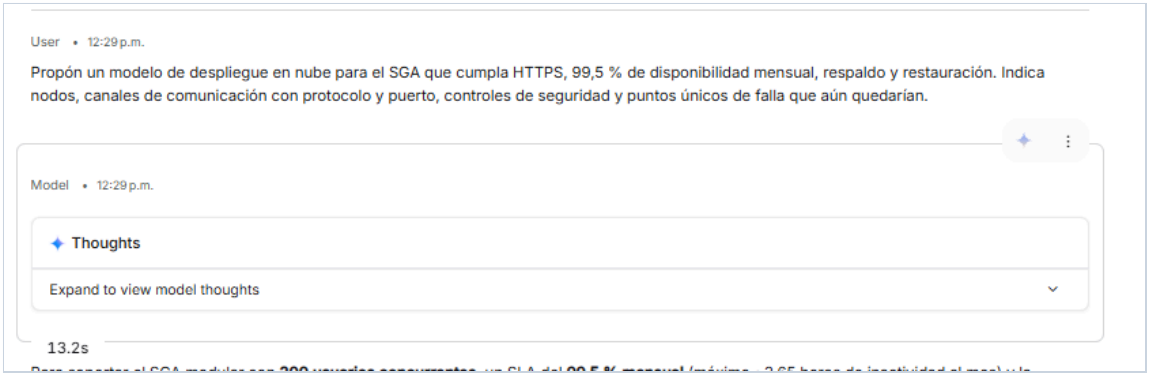


Evidencia 12. Secuencia de matrícula concurrente con `SELECT … FOR UPDATE` (parte 1).

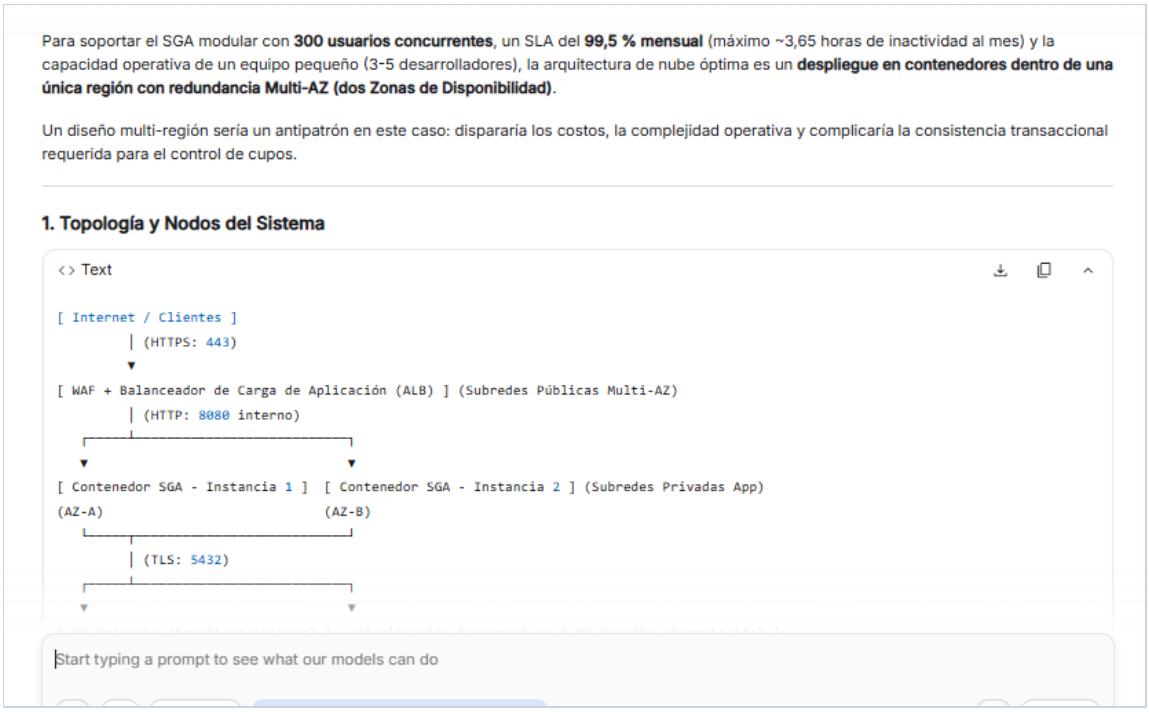


Evidencia 13. Secuencia de matrícula concurrente (parte 2): auditoría registrada después del COMMIT.

Prompt 4 — Despliegue y seguridad



Evidencia 14. Prompt 4.



Para soportar el SGA modular con **300 usuarios concurrentes**, un SLA del **99,5 % mensual** (máximo ~3,65 horas de inactividad al mes) y la capacidad operativa de un equipo pequeño (3-5 desarrolladores), la arquitectura de nube óptima es un **despliegue en contenedores dentro de una única región con redundancia Multi-AZ (dos Zonas de Disponibilidad)**.

Un diseño multi-región sería un antipatrón en este caso: dispararía los costos, la complejidad operativa y complicaría la consistencia transaccional requerida para el control de cupos.

1. Topología y Nodos del Sistema

Text

```
[ Internet / Clientes ]
        | (HTTPS: 443)
        ▼
[ WAF + Balanceador de Carga de Aplicación (ALB) ] (Subredes Públicas Multi-AZ)
        | (HTTP: 8080 interno)
   ┌────┴──────────────────────────┐
   ▼                               ▼
[ Contenedor SGA - Instancia 1 ]  [ Contenedor SGA - Instancia 2 ] (Subredes Privadas App)
(AZ-A)                            (AZ-B)
   └────┬──────────────────────────┘
        | (TLS: 5432)
   ┌────┴──────────────────────────┐
   ▼                               ▼
```

Start typing a prompt to see what our models can do

Evidencia 15. Propuesta: una región con dos zonas de disponibilidad; topología.



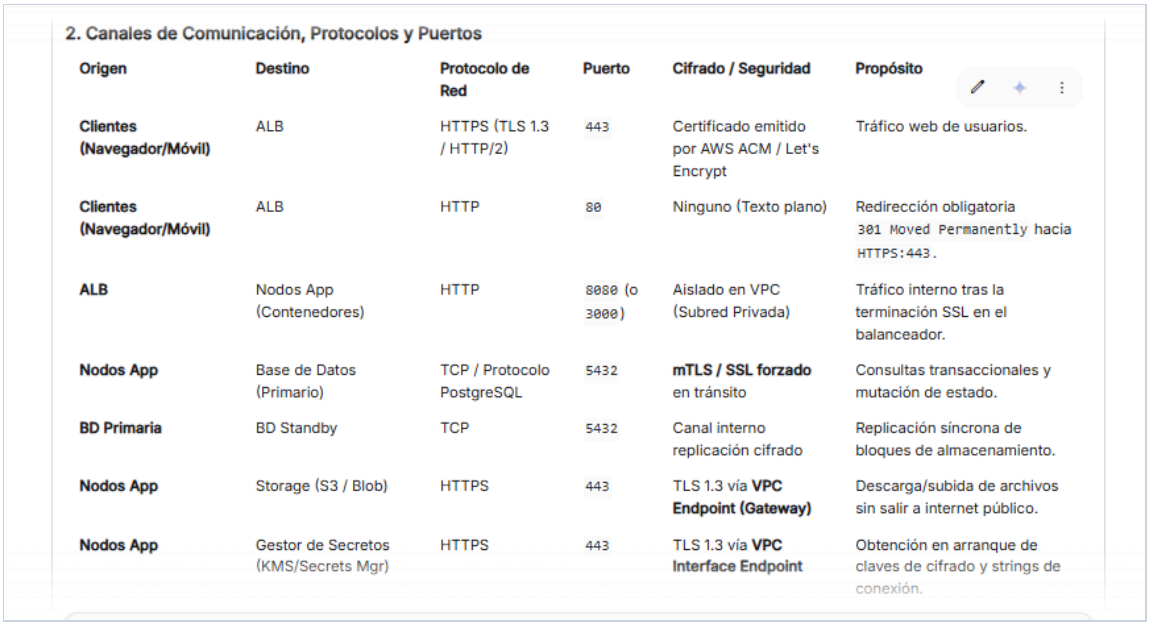
Text

```
(AZ-A) Subred Privada Datos                    (AZ-B) Subred Privada Datos
```

- **Capa de Borde y Balanceo (Subredes Públicas en AZ-A y AZ-B):**
  - **Application Load Balancer (ALB):** Recibe el tráfico exterior, gestiona el certificado SSL/TLS y distribuye la carga equitativamente hacia los contenedores.
- **Capa de Cómputo (Subredes Privadas en AZ-A y AZ-B):**
  - **2 Nodos de Aplicación (ECS Fargate / Google Cloud Run / VMs):** Mínimo 2 contenedores del monolito modular activos permanentemente (uno por cada AZ). Para 300 usuarios concurrentes, 2 tareas de 1–2 vCPU y 2–4 GB RAM cada una brindan redundancia $N+1$ con holgura.
- **Capa de Datos (Subredes Privadas de Datos en AZ-A y AZ-B):**
  - **Base de Datos Relacional Gestionada (ej. AWS RDS PostgreSQL / Aurora Multi-AZ):**
    - **Nodo Primario (AZ-A):** Atiende lecturas y escrituras transaccionales con bloqueo pesimista para cupos.
    - **Nodo Réplica Standby (AZ-B):** Réplica pasiva con sincronización síncrona. Si falla AZ-A, el motor efectúa *failover* automático (conmutación de DNS) en 60–120 segundos.
- **Almacenamiento de Objetos (Servicio Global/Regional Gestionado):**
  - **Object Storage (S3 / Cloud Storage):** Almacenamiento fuera de banda de documentos, fotos, sílabos y backups lógicos.

2. Canales de Comunicación, Protocolos y Puertos

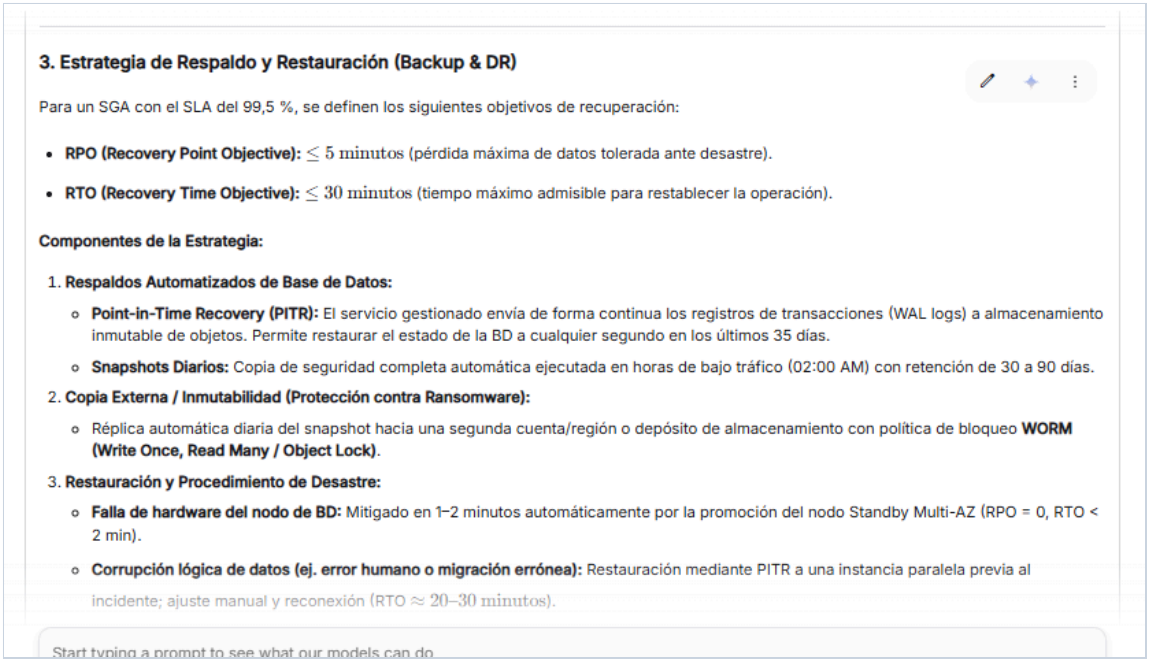
Evidencia 16. Nodos: balanceador, dos contenedores, PostgreSQL con standby síncrono, almacenamiento de objetos.



2. Canales de Comunicación, Protocolos y Puertos

Origen	Destino	Protocolo de Red	Puerto	Cifrado / Seguridad	Propósito
Clientes (Navegador/Móvil)	ALB	HTTPS (TLS 1.3 / HTTP/2)	`443`	Certificado emitido por AWS ACM / Let's Encrypt	Tráfico web de usuarios.
Clientes (Navegador/Móvil)	ALB	HTTP	`80`	Ninguno (Texto plano)	Redirección obligatoria `301 Moved Permanently` hacia `HTTPS:443`.
ALB	Nodos App (Contenedores)	HTTP	`8080` (o `3000`)	Aislado en VPC (Subred Privada)	Tráfico interno tras la terminación SSL en el balanceador.
Nodos App	Base de Datos (Primario)	TCP / Protocolo PostgreSQL	`5432`	**mTLS / SSL forzado** en tránsito	Consultas transaccionales y mutación de estado.
BD Primaria	BD Standby	TCP	`5432`	Canal interno replicación cifrado	Replicación síncrona de bloques de almacenamiento.
Nodos App	Storage (S3 / Blob)	HTTPS	`443`	TLS 1.3 vía **VPC Endpoint (Gateway)**	Descarga/subida de archivos sin salir a internet público.
Nodos App	Gestor de Secretos (KMS/Secrets Mgr)	HTTPS	`443`	TLS 1.3 vía **VPC Interface Endpoint**	Obtención en arranque de claves de cifrado y strings de conexión.

Evidencia 17. Canales de comunicación, protocolos y puertos.



3. Estrategia de Respaldo y Restauración (Backup & DR)

Para un SGA con el SLA del 99,5 %, se definen los siguientes objetivos de recuperación:

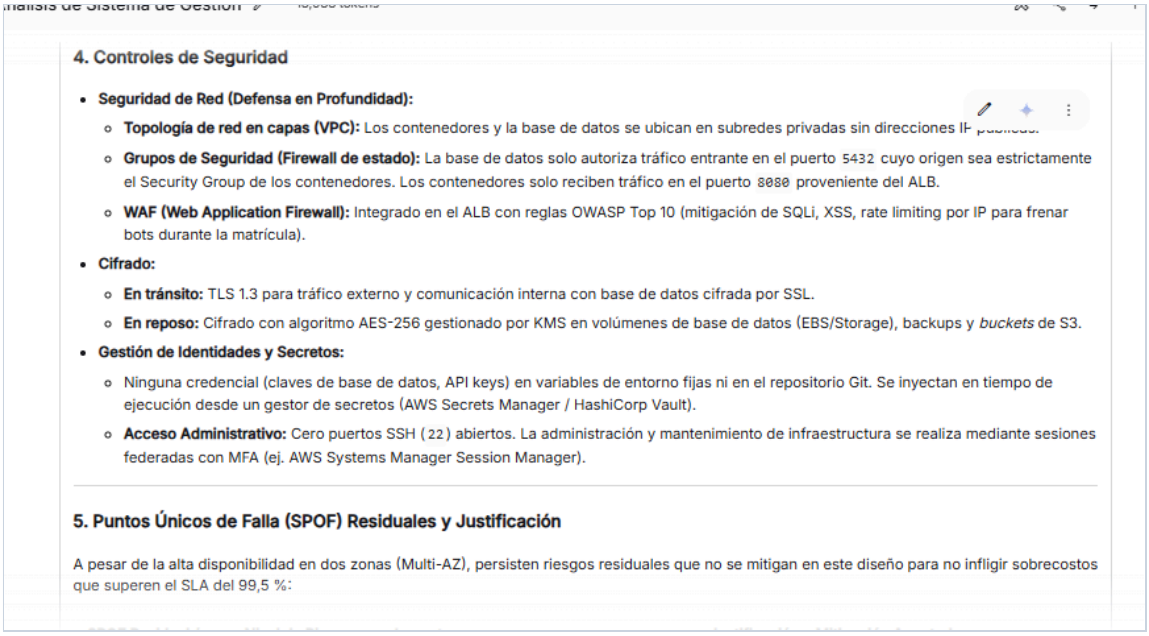
- **RPO (Recovery Point Objective):** ≤ 5 minutos (pérdida máxima de datos tolerada ante desastre).
- **RTO (Recovery Time Objective):** ≤ 30 minutos (tiempo máximo admisible para restablecer la operación).

Componentes de la Estrategia:

1. **Respaldos Automatizados de Base de Datos:**
   - **Point-in-Time Recovery (PITR):** El servicio gestionado envía de forma continua los registros de transacciones (WAL logs) a almacenamiento inmutable de objetos. Permite restaurar el estado de la BD a cualquier segundo en los últimos 35 días.
   - **Snapshots Diarios:** Copia de seguridad completa automática ejecutada en horas de bajo tráfico (02:00 AM) con retención de 30 a 90 días.
2. **Copia Externa / Inmutabilidad (Protección contra Ransomware):**
   - Réplica automática diaria del snapshot hacia una segunda cuenta/región o depósito de almacenamiento con política de bloqueo **WORM (Write Once, Read Many / Object Lock)**.
3. **Restauración y Procedimiento de Desastre:**
   - **Falla de hardware del nodo de BD:** Mitigado en 1–2 minutos automáticamente por la promoción del nodo Standby Multi-AZ (RPO = 0, RTO < 2 min).
   - **Corrupción lógica de datos (ej. error humano o migración errónea):** Restauración mediante PITR a una instancia paralela previa al incidente; ajuste manual y reconexión (RTO $\approx$ 20–30 minutos).

Start typing a prompt to see what our models can do

Evidencia 18. Respaldo y restauración: RPO ≤ 5 min, RTO ≤ 30 min, PITR, copias inmutables (WORM).



4. Controles de Seguridad

- **Seguridad de Red (Defensa en Profundidad):**
  - **Topología de red en capas (VPC):** Los contenedores y la base de datos se ubican en subredes privadas sin direcciones IP
  - **Grupos de Seguridad (Firewall de estado):** La base de datos solo autoriza tráfico entrante en el puerto `5432` cuyo origen sea estrictamente el Security Group de los contenedores. Los contenedores solo reciben tráfico en el puerto `8080` proveniente del ALB.
  - **WAF (Web Application Firewall):** Integrado en el ALB con reglas OWASP Top 10 (mitigación de SQLi, XSS, rate limiting por IP para frenar bots durante la matrícula).
- **Cifrado:**
  - **En tránsito:** TLS 1.3 para tráfico externo y comunicación interna con base de datos cifrada por SSL.
  - **En reposo:** Cifrado con algoritmo AES-256 gestionado por KMS en volúmenes de base de datos (EBS/Storage), backups y *buckets* de S3.
- **Gestión de Identidades y Secretos:**
  - Ninguna credencial (claves de base de datos, API keys) en variables de entorno fijas ni en el repositorio Git. Se inyectan en tiempo de ejecución desde un gestor de secretos (AWS Secrets Manager / HashiCorp Vault).
  - **Acceso Administrativo:** Cero puertos SSH (`22`) abiertos. La administración y mantenimiento de infraestructura se realiza mediante sesiones federadas con MFA (ej. AWS Systems Manager Session Manager).

---

5. Puntos Únicos de Falla (SPOF) Residuales y Justificación

A pesar de la alta disponibilidad en dos zonas (Multi-AZ), persisten riesgos residuales que no se mitigan en este diseño para no infligir sobrecostos que superen el SLA del 99,5 %:

Evidencia 19. Controles de seguridad: red, cifrado, secretos y acceso administrativo con MFA.



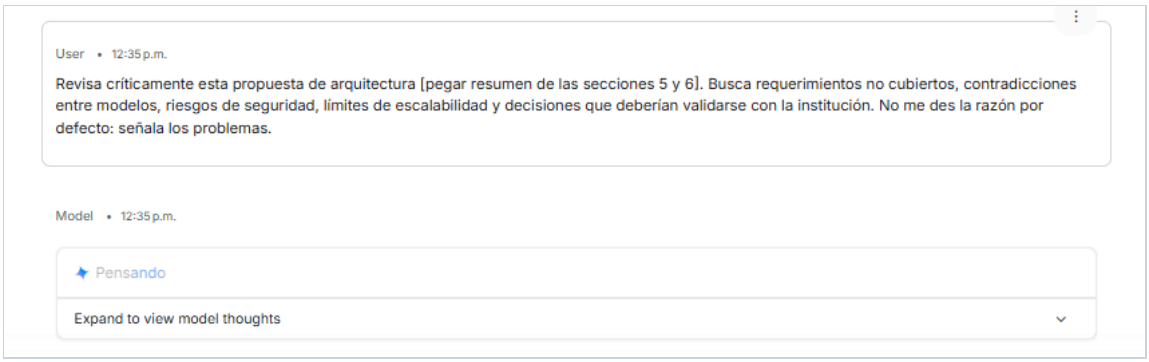
5. Puntos Únicos de Falla (SPOF) Residuales y Justificación

A pesar de la alta disponibilidad en dos zonas (Multi-AZ), persisten riesgos residuales que no se mitigan en este diseño para no s que superen el SLA del 99,5 %:

SPOF Residual / Escenario de Falla	Nivel de Riesgo	Impacto	Justificación y Mitigación Aceptada
Caída Total de la Región Cloud *(Desastre regional mayor)*	Extremadamente Bajo	Indisponibilidad total del servicio hasta que la región se recupere.	**Aceptado:** Una caída completa de región ocurre con una frecuencia menor a una vez cada varios años. Implementar multi-región activa aumentaría el presupuesto por un factor de 3x a 4x y rompería la transacción ACID unificada para cupos.
Instancia Primaria de BD durante el *Failover*	Bajo	Durante la caída del nodo primario, transcurren entre **60 y 120 segundos** mientras se promueve el Standby y conmuta el DNS interno. En ese lapso, la matrícula falla.	**Aceptado:** 2 minutos de caída eventual están muy por debajo del margen permitido por el SLA de 99,5 % (hasta 219 minutos/mes). La aplicación reintenta la conexión automáticamente.
Falla en el Servicio de DNS / Autoridad de Dominio	Muy Bajo	Si el proveedor DNS (ej. Route 53 o registrador) falla, los clientes no resuelven el nombre de dominio.	**Mitigado parcialmente:** Configurar TTLs razonables (300 s) y usar proveedores con Anycast global de alta disponibilidad (SLA 100 %).
Corrupción Lógica por Despliegue	Medio	Un script de migración SQL erróneo o un fallo en el monolito que escape a las	**Mitigación Operativa:** Uso de despliegues tipo *Rolling Update* con *Health Checks* estrictos y respaldos

Evidencia 20. Puntos únicos de falla residuales.

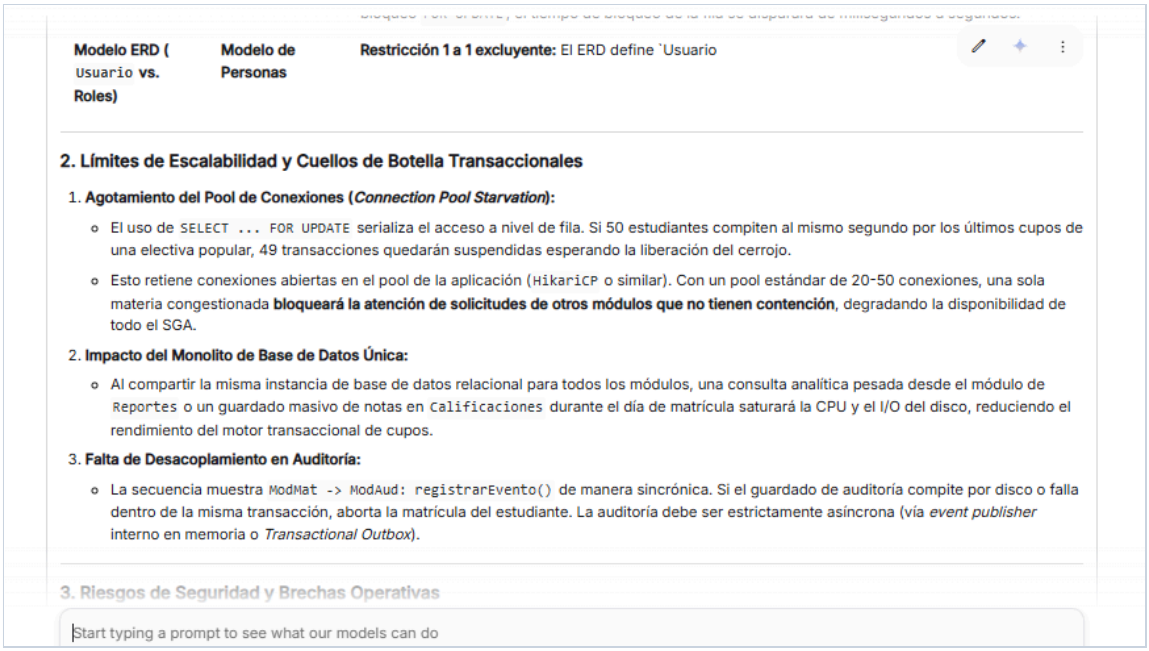
Prompt 5 — Revisión crítica



Evidencia 21. Prompt 5.



Evidencia 22. Contradicciones entre modelos detectadas por la IA.



Modelo ERD (`Usuario` vs. Roles)	Modelo de Personas	**Restricción 1 a 1 excluyente:** El ERD define `Usuario

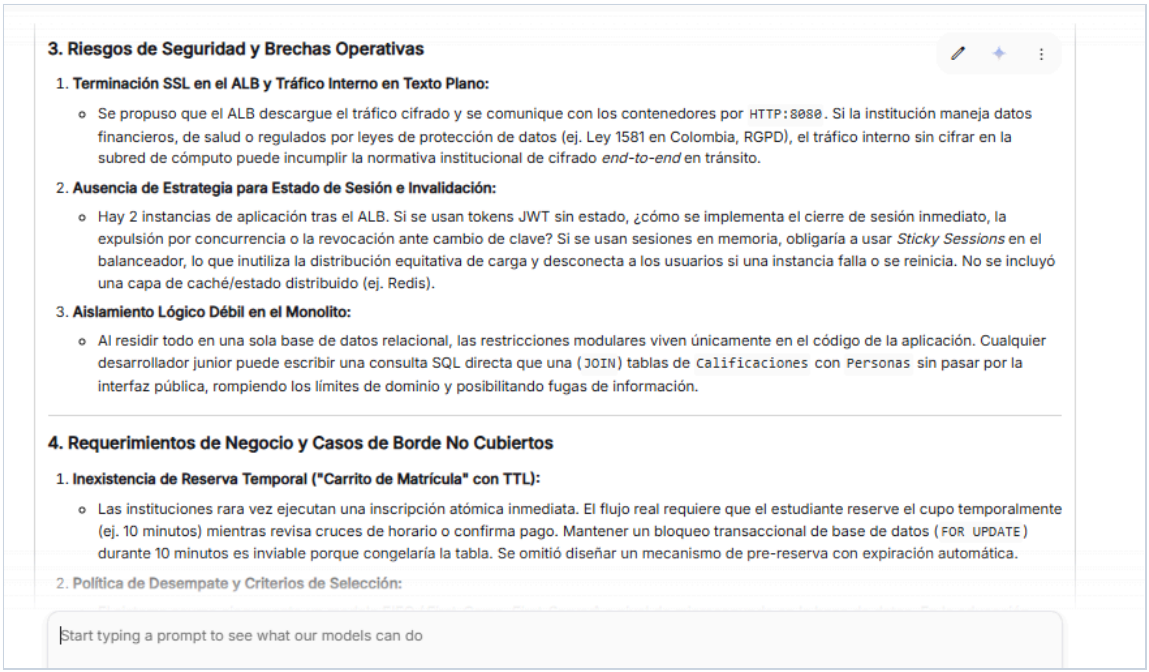
2. Límites de Escalabilidad y Cuellos de Botella Transaccionales

1. **Agotamiento del Pool de Conexiones (*Connection Pool Starvation*):**
   - El uso de `SELECT ... FOR UPDATE` serializa el acceso a nivel de fila. Si 50 estudiantes compiten al mismo segundo por los últimos cupos de una electiva popular, 49 transacciones quedarán suspendidas esperando la liberación del cerrojo.
   - Esto retiene conexiones abiertas en el pool de la aplicación (`HikariCP` o similar). Con un pool estándar de 20-50 conexiones, una sola materia congestionada **bloqueará la atención de solicitudes de otros módulos que no tienen contención**, degradando la disponibilidad de todo el SGA.
2. **Impacto del Monolito de Base de Datos Única:**
   - Al compartir la misma instancia de base de datos relacional para todos los módulos, una consulta analítica pesada desde el módulo de `Reportes` o un guardado masivo de notas en `Calificaciones` durante el día de matrícula saturará la CPU y el I/O del disco, reduciendo el rendimiento del motor transaccional de cupos.
3. **Falta de Desacoplamiento en Auditoría:**
   - La secuencia muestra `ModMat -> ModAud: registrarEvento()` de manera sincrónica. Si el guardado de auditoría compite por disco o falla dentro de la misma transacción, aborta la matrícula del estudiante. La auditoría debe ser estrictamente asíncrona (vía *event publisher* interno en memoria o *Transactional Outbox*).

3. Riesgos de Seguridad y Brechas Operativas

Start typing a prompt to see what our models can do

Evidencia 23. Límites de escalabilidad: agotamiento del pool de conexiones, base de datos compartida, auditoría síncrona.



3. Riesgos de Seguridad y Brechas Operativas

1. **Terminación SSL en el ALB y Tráfico Interno en Texto Plano:**
   - Se propuso que el ALB descargue el tráfico cifrado y se comunique con los contenedores por `HTTP:8080`. Si la institución maneja datos financieros, de salud o regulados por leyes de protección de datos (ej. Ley 1581 en Colombia, RGPD), el tráfico interno sin cifrar en la subred de cómputo puede incumplir la normativa institucional de cifrado *end-to-end* en tránsito.
2. **Ausencia de Estrategia para Estado de Sesión e Invalidación:**
   - Hay 2 instancias de aplicación tras el ALB. Si se usan tokens JWT sin estado, ¿cómo se implementa el cierre de sesión inmediato, la expulsión por concurrencia o la revocación ante cambio de clave? Si se usan sesiones en memoria, obligaría a usar *Sticky Sessions* en el balanceador, lo que inutiliza la distribución equitativa de carga y desconecta a los usuarios si una instancia falla o se reinicia. No se incluyó una capa de caché/estado distribuido (ej. Redis).
3. **Aislamiento Lógico Débil en el Monolito:**
   - Al residir todo en una sola base de datos relacional, las restricciones modulares viven únicamente en el código de la aplicación. Cualquier desarrollador junior puede escribir una consulta SQL directa que una (`JOIN`) tablas de `Calificaciones` con `Personas` sin pasar por la interfaz pública, rompiendo los límites de dominio y posibilitando fugas de información.

4. Requerimientos de Negocio y Casos de Borde No Cubiertos

1. **Inexistencia de Reserva Temporal ("Carrito de Matrícula" con TTL):**
   - Las instituciones rara vez ejecutan una inscripción atómica inmediata. El flujo real requiere que el estudiante reserve el cupo temporalmente (ej. 10 minutos) mientras revisa cruces de horario o confirma pago. Mantener un bloqueo transaccional de base de datos (`FOR UPDATE`) durante 10 minutos es inviable porque congelaría la tabla. Se omitió diseñar un mecanismo de pre-reserva con expiración automática.
2. Política de Desempate y Criterios de Selección:

Start typing a prompt to see what our models can do

Evidencia 24. Riesgos de seguridad y casos de borde no cubiertos.



2. **Política de Desempate y Criterios de Selección:**
   - El sistema asume ciegamente un modelo FIFO (*First-Come, First-Served*) a nivel de microsegundo en la base de datos. En superior, los cupos suelen asignarse mediante **turnos de matrícula escalonados** (por promedio ponderado, semestre o mérito). La arquitectura carece de un motor de validación de ventanas temporales por perfil.
3. **Manejo de Falla Parcial en la Matrícula:**
   - Si un estudiante postula a 6 materias y solo 1 no tiene cupo: ¿el sistema rechaza toda la matrícula (rollback total) o inscribe las 5 restantes e informa el conflicto (éxito parcial)? El diseño actual no define la semántica ante fallos parciales.

---

5. Decisiones Críticas que DEBEN Validarse con la Institución

Antes de aprobar esta arquitectura, el equipo debe plantear formalmente a los directivos y stakeholders de la universidad los siguientes puntos:

1. **Autenticación Institucional vs. Módulo IAM Propio:**
   - ¿La universidad realmente necesita un módulo de identidad desde cero? Diseñar credenciales locales suele ser un error cuando el 95 % de las instituciones ya cuenta con **Microsoft 365 (Entra ID)** o **Google Workspace**. Es imperativo validar si el módulo IAM debe convertirse simplemente en un adaptador de **SSO (Single Sign-On vía SAML 2.0 / OIDC)**.
2. **Definición Real del SLA del 99,5 %:**
   - ¿Este 99,5 % incluye o excluye las ventanas de mantenimiento programadas? Si las incluye, cualquier despliegue que requiera migraciones de base de datos con bloqueo de tablas consumirá gran parte del presupuesto de error mensual (~3,6 horas).
3. **Presupuesto Mensual de Infraestructura Cloud:**
   - Una arquitectura Multi-AZ redundante en AWS (ALB + 2 tareas Fargate + RDS Multi-AZ + NAT Gateways + KMS + WAF) tiene un costo base aproximado de **$250 a $450 USD/mes**. Se debe validar si la institución dispone de esta partida operativa o si espera reutilizar servidores físicos (*on-premise*).

Start typing a prompt to see what our models can do

Evidencia 25. Decisiones que deben validarse con la institución.

9.3 Sugerencias aceptadas, modificadas o descartadas

Tema (evidencia)	Qué propuso la IA	Decisión	Justificación
Análisis de requerimientos (Ev. 2)	No hizo el análisis porque faltaba el texto; pidió el enunciado y propuso una estructura: tablas de actores, supuestos marcados, restricciones y reglas de negocio; criterios de aceptación SMART; requerimientos significativos para la arquitectura (ASR).	Aceptada	Se adoptó esa estructura en las secciones 3 y 4 (supuestos con identificador, criterio de aceptación por requerimiento, requerimientos que dirigen la arquitectura). El contenido proviene del enunciado, no de la IA.
Estilo arquitectónico (Ev. 4–8)	Monolito modular, por el tamaño del equipo (ley de Conway), la carga de 300 usuarios y la consistencia ACID de los cupos; microservicios solo si aparecen más de 15–20 desarrolladores, picos de más de 10 000 req/s o requisitos regulatorios propios de un módulo.	Aceptada	Coincide con la evaluación propia de la sección 5.1, hecha con criterios del enunciado. Se acepta por esa evidencia y no por la recomendación. Los umbrales de cambio se incorporaron a la sección 8.5.
Control de cupos (Ev. 6, 12, 13)	Dos variantes: `UPDATE … SET cupos = cupos − 1 WHERE cupos > 0` (Ev. 6) y bloqueo pesimista `SELECT … FOR UPDATE` (Ev. 12).	Modificada	Ambas impiden el sobrecupo. Se mantiene el bloqueo explícito porque en la misma transacción también se valida la duplicidad y se escribe la auditoría; la actualización condicional se documenta como alternativa equivalente. Se agregó el índice único parcial como segunda barrera, que ninguna variante de la IA incluía.
Auditoría en la secuencia (Ev. 13, 23)	La secuencia del prompt 3 registra el evento de auditoría después del COMMIT. En el prompt 5, la misma IA advierte que una auditoría síncrona puede abortar la matrícula y recomienda un *transactional outbox*.	Modificada	Registrar después del COMMIT puede perder el evento si el proceso falla entre los dos pasos (incumple RF-12). Se adopta el outbox dentro de la misma transacción (ADR-04), que es lo que la IA terminó recomendando.
Modelo de datos (Ev. 11, 22)	Matrícula como encabezado con varias filas de MatriculaDetalle (una por asignatura).	Modificada	Se usa una matrícula por grupo: simplifica el índice único (RN-01) y la secuencia. La IA misma señaló (Ev. 22) que su modelo era inconsistente con su secuencia de un solo grupo. Se agregaron PlanEstudio, Permiso y Horario.
Diagramas (Ev. 10–13)	Código Mermaid de componentes, entidad-relación y secuencia.	Modificada	El diagrama de componentes resultó ilegible por su ancho (Ev. 10). Los diagramas finales se rehicieron en Graphviz y Mermaid, con nombres consistentes con el resto del informe.

Tema (evidencia)	Qué propuso la IA	Decisión	Justificación
Despliegue (Ev. 15–17)	Una región con dos zonas, ALB con WAF, dos contenedores, PostgreSQL con standby síncrono; calificó la arquitectura multirregión activa como antipatrón por costo.	Aceptada	Coincide con la figura 3. Solo los respaldos se copian a otra región; no hay multirregión activa.
Tráfico interno (Ev. 17, 24)	El prompt 4 propone HTTP:8080 sin cifrar entre balanceador y contenedores; en el prompt 5 la IA lo señala como riesgo frente a la Ley 1581.	Modificada	Se cambió a TLS también en el tramo balanceador → API (re-cifrado). El costo es menor y protege datos personales en tránsito (RNF-11).
RPO/RTO (Ev. 18)	RPO ≤ 5 min y RTO ≤ 30 min, con PITR y copias inmutables (WORM).	Modificada	Se adoptan las copias inmutables. Para RPO y RTO se mantienen ≤ 15 min y ≤ 4 h como compromiso verificable, porque un RTO de 30 min exige ensayos y operación que la institución debe confirmar (sección 8.6).
Concurrencia y pool (Ev. 22, 23)	Interbloqueos si un estudiante matricula varios grupos a la vez, y agotamiento del pool de conexiones mientras esperan los bloqueos.	Aceptada	Hallazgo válido no cubierto antes. Se incorporó a ADR-02 y al riesgo R11: bloqueo en orden determinista por id de grupo, `lock_timeout` corto y límite de concurrencia por grupo.
Casos de borde (Ev. 24)	Reserva temporal de cupos («carrito»), turnos de matrícula escalonados y semántica ante fallo parcial.	Modificada	No están en los RF del enunciado. Se definió que cada grupo se procesa de forma independiente (éxito parcial informado por asignatura); la reserva temporal y los turnos quedan como decisiones a validar (8.6).
SLA y mantenimientos (Ev. 25)	Pregunta si el 99,5 % incluye las ventanas de mantenimiento.	Descartada	El enunciado (RNF-03) ya excluye los mantenimientos programados e informados, así que no es un punto abierto.
Identidad y costo (Ev. 25)	Validar si se usa SSO institucional (Entra ID / Google Workspace) y estimar el costo de nube (USD 250–450/mes).	Aceptada	Ya figuraban como decisiones a validar (8.6). La cifra de costo es una estimación de la IA, no verificada.

9.4 Lecciones sobre el uso de la IA

- La herramienta acelera la comparación de alternativas y los borradores, pero se contradijo a sí misma entre prompts (auditoría síncrona en el prompt 3, outbox en el prompt 5; HTTP interno en el prompt 4, TLS en el prompt 5). Revisar cada respuesta contra las demás fue indispensable.
- La revisión crítica (prompt 5) fue la más valiosa: encontró dos problemas reales (interbloqueos y agotamiento del pool) que se incorporaron al diseño.
- Dejar marcadores sin reemplazar en un prompt degrada la respuesta; la IA trabajó sin el texto de los requerimientos. Por eso los requerimientos del informe provienen del enunciado y no de la IA.
- La responsabilidad sobre la exactitud, la coherencia y la viabilidad de la propuesta es del autor; cada decisión final tiene su propio ADR.

10. Video pitch y repositorio

Video pitch (YouTube)	https://youtu.be/Ch0j0luFkEQ
Repositorio	https://github.com/jhonnathancun/sga

Contenido del video (5 a 7 minutos): problema y objetivo (30 s); requerimientos que dirigen la arquitectura (1 min); estilo seleccionado y comparación (1 min 30 s); recorrido por componentes, despliegue y secuencia de matrícula (2 min); decisiones y riesgos principales (1 min); uso crítico de Google AI Studio y conclusiones (1 min). Presentado por el autor del trabajo.

Contenido del repositorio:

```
sga/
├── README.md                    Descripción, índice y enlaces
├── informe/
│   └── ACA_Arquitectura_SGA.pdf  Este informe
├── diagramas/
│   ├── 01_casos_de_uso.dot / .svg / .png
│   ├── 02_componentes.dot  / .svg / .png
│   ├── 03_despliegue.dot   / .svg / .png
│   ├── 04_modelo_datos.dot  / .svg / .png
│   ├── 05_secuencia_matricula.mmd     / .svg / .png
│   └── 06_secuencia_calificacion.mmd  / .svg / .png
├── adr/                         ADR-01 a ADR-06 en Markdown
└── evidencias-ai-studio/        Capturas de prompts y respuestas
```

Para regenerar los diagramas: `dot -Tsvg archivo.dot -o archivo.svg` (Graphviz) y `mmdc -i archivo.mmd -o archivo.svg` (Mermaid CLI).

11. Conclusiones y recomendaciones

11.1 Conclusiones

- Un **monolito modular con arquitectura limpia por módulo** es el estilo más adecuado para el SGA: obtuvo la mejor evaluación frente a los atributos de calidad, garantiza consistencia en matrícula y calificaciones con transacciones locales y mantiene bajo acoplamiento sin el costo operativo de los microservicios.
- Los requerimientos que más condicionan el diseño son la integridad ante concurrencia (RNF-07), la seguridad y privacidad por rol (RNF-01, RNF-11), la disponibilidad (RNF-03) y la auditoría (RF-12). Cada uno tiene una decisión explícita (ADR-02 a ADR-05) y un método de verificación.
- La matriz de trazabilidad cubre los 24 requerimientos; ningún componente existe sin un requerimiento que lo justifique.
- El punto más crítico de la propuesta no es la API, que escala horizontalmente, sino la base de datos de escritura durante la apertura de matrícula; por eso se priorizó su prueba de carga y se definió una ruta de evolución.

- Google AI Studio fue útil para explorar alternativas y producir borradores, pero varias sugerencias requirieron corrección, especialmente en concurrencia, autenticación y auditoría. El valor estuvo en verificar, no en copiar.

11.2 Recomendaciones

1. Construir primero un **prototipo vertical** del flujo de matrícula (SPA → API → BD → auditoría) y ejecutar la prueba de concurrencia y de carga antes de desarrollar los demás módulos.
2. Incorporar desde el primer día las **pruebas de arquitectura** en la integración continua para que las fronteras entre módulos no se erosionen.
3. Validar con la institución los puntos de la sección 8.6, en especial RPO/RTO y la existencia de un proveedor de identidad.
4. Planear la **migración de datos** como un proyecto aparte, con deduplicación y validación, porque los datos actuales son la causa del problema.
5. Exigir **segundo factor de autenticación** para administradores y coordinadores.
6. Realizar un **simulacro de restauración** por periodo académico y conservar el acta como evidencia de RNF-09.

12. Referencias

Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4th ed.). Addison-Wesley.

Brown, S. (s.f.). *The C4 model for visualising software architecture*. https://c4model.com

Congreso de Colombia. (2012, 17 de octubre). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial No. 48.587.

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Fowler, M. (2015, 3 de junio). *MonolithFirst*. martinfowler.com. https://martinfowler.com/bliki/MonolithFirst.html

Google. (s.f.). *Google AI Studio*. https://aistudio.google.com

International Organization for Standardization. (2023). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model* (ISO/IEC 25010:2023).

International Organization for Standardization. (2022). *Software, systems and enterprise — Architecture description* (ISO/IEC/IEEE 42010:2022).

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Newman, S. (2019). *Monolith to microservices: Evolutionary patterns to transform your monolith*. O'Reilly Media.

OWASP Foundation. (2021). *OWASP Application Security Verification Standard 4.0.3*. https://owasp.org/www-project-application-security-verification-standard/

OWASP Foundation. (s.f.). *Password storage cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

PostgreSQL Global Development Group. (s.f.). *Continuous archiving and point-in-time recovery (PITR)*. PostgreSQL documentation. https://www.postgresql.org/docs/current/continuous-archiving.html

Richards, M., & Ford, N. (2020). *Fundamentals of software architecture: An engineering approach*. O'Reilly Media.

Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Pearson Educación.

World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/